

Diseño e Implementación de un Brazo Robótico de 3 GDL para Corte Láser No-Coplanar

Ivan Yerry Hinojosa Zegarra
EAP Ingeniería Mecatrónica
Universidad Continental
Huancayo, Perú
72241950@continental.edu.pe

Brayan Omar Duran Arroyo
EAP Ingeniería Mecatrónica
Universidad Continental
Huancayo, Perú
72542899@continental.edu.pe

Jorge Andre Tapia Patiño
EAP Ingeniería Mecatrónica
Universidad Continental
Huancayo, Perú
60761518@continental.edu.pe

Carlos Benedic Sosa Carhuamaca
EAP Ingeniería Mecatrónica
Universidad Continental
Huancayo, Perú
60847160@continental.edu.pe

Resumen—El presente artículo detalla el diseño, simulación y construcción física de un brazo manipulador antropomórfico de 3 grados de libertad (GDL) orientado al corte y perfilado láser no-coplanar. El objetivo principal ha sido desarrollar un sistema capaz de seguir trayectorias tridimensionales sobre superficies cilíndricas e irregulares mediante un emisor láser en su efector final, controlado por un microcontrolador ESP32. Se han implementado los modelos de cinemática directa, inversa y Jacobiana para garantizar el posicionamiento espacial en los tres ejes (X, Y, Z) y conservar la distancia focal del láser. La estructura física se fabricó en impresión 3D a partir de mallas STL, con un firmware desarrollado en C++. La validación cinemática se realizó mediante simulación virtual en ROS 2 y Gazebo de forma puramente offline.

Index Terms—Cinemática, brazo robótico, ESP32, C++, ROS 2, simulación virtual.

I. INTRODUCCIÓN

En la industria moderna, los brazos robóticos son herramientas fundamentales para tareas de precisión como la soldadura y el corte láser. Comprender la cinemática detrás de estos mecanismos es el primer paso para su control y programación [1]. Este proyecto busca aplicar los conceptos teóricos de modelado de robots —tales como matrices de transformación homogénea, representación de orientación y cinemática diferencial— en un caso de uso práctico.

Al estructurar el modelo con 3 grados de libertad (GDL) (rotación en la base, elevación en el hombro y elevación en el codo), se simplificó el análisis geométrico sin perder la capacidad de posicionar el efector final en un volumen de trabajo tridimensional. Además, el proyecto utiliza un emisor láser de baja potencia (Clase 1/2) como efector final. A diferencia de procesos de contacto como la soldadura por arco, el láser permite una validación cinemática de “no-contacto”. Esto simplifica el control al eliminar la necesidad de compensar fuerzas de fricción o colisiones mecánicas directas contra la pieza de trabajo, permitiendo un seguimiento de trayectoria preciso y eficiente. Este enfoque es crucial para

el corte y grabado sobre superficies irregulares y curvadas en tres dimensiones, donde el brazo debe seguir contornos tridimensionales curvos sobre una superficie no plana manteniendo constante la distancia focal del haz láser, una operación inviable para máquinas cartesianas 2D tradicionales. El desarrollo del software, simulación, firmware embebido e interfaz web se encuentra disponible de forma pública en el repositorio de GitHub del proyecto [2].

II. OBJETIVOS

A. Objetivo General

Desarrollar, simular y construir un brazo robótico articulado de 3 GDL orientado a procesos de corte láser no-coplanar sobre superficies curvas e irregulares, validando los modelos cinemáticos en simulación virtual (ROS 2/Gazebo) y, de forma independiente, en el prototipo físico controlado por un ESP32.

B. Objetivos Específicos

- Definir los parámetros de Denavit-Hartenberg y resolver analíticamente la cinemática directa, inversa y Jacobiana para garantizar el seguimiento espacial tridimensional (X, Y, Z) y la evasión de singularidades en superficies no planas.
- Modelar la estructura física en URDF/Xacro y simular el comportamiento del sistema utilizando el sistema operativo de robots (ROS 2, por sus siglas en inglés) y Gazebo de manera exclusivamente virtual, sin conexión física con el prototipo.
- Fabricar los eslabones y la base del robot utilizando tecnologías de impresión 3D a partir de archivos STL.
- Implementar el control de bajo nivel del hardware mediante un microcontrolador ESP32, utilizando C++ para accionar las articulaciones físicas del prototipo.

III. MODELADO CINEMÁTICO

Se utiliza la convención de Denavit-Hartenberg (D-H) clásica para describir la geometría y cinemática del brazo manipulador antropomórfico de 3 GDL.

A. Justificación de la Morfología y Comparación de Alternativas

El brazo robótico desarrollado se destina a la aplicación de corte y perfilado láser no-coplanar sobre piezas cilíndricas. Específicamente, el objetivo es seguir trayectorias espaciales complejas sobre una superficie cilíndrica con un diámetro real de 5,3 cm (radio $R_{\text{tubo}} = 26,5$ mm) montada sobre un soporte de 60 mm de altura. La pieza cilíndrica está posicionada a una distancia nominal de $x_{\text{tubo}} = 350$ mm y una altura de base $z_{\text{tubo}} = 60$ mm con respecto al origen de la base del robot. Para esta tarea, es indispensable posicionar el efector final de manera tridimensional (X, Y, Z) controlando dinámicamente la altura del cabezal para mantener una distancia de enfoque constante normal al cilindro, evitando colisiones con la estructura del tubo.

Las restricciones físicas del proyecto imponen el uso de actuadores servo de bajo costo (dos MG996R y un MG90S) con un presupuesto de torque limitado. Esto restringe la masa del efector final, lo que justifica la selección de un diodo láser semiconductor ligero de estado sólido (10 g) que no introduce esfuerzos inerciales significativos a diferencia de herramientas de mecanizado por contacto.

Con el fin de justificar la selección de la morfología antropomórfica de 3 GDL (RRR), se analizaron diferentes arquitecturas robóticas estándar de 3 GDL (Tabla I).

Tabla I
COMPARACIÓN DE MORFOLOGÍAS DE 3 GDL PARA CORTE LÁSER DE PIEZAS CILÍNDRICAS

Morfología	Config.	Ventajas	Desventajas
Cartesiana	PPP	Alta rigidez mecánica, linealidad directa en X, Y, Z .	Gran volumen ocupado (footprint), incapacidad de esquivar obstáculos sin colisionar lateralmente.
SCARA	RRP	Alta velocidad en plano horizontal.	Carrera vertical (Z) reducida para geometrías curvas con gran pendiente vertical.
Cilíndrica / Polar	RPP / RRP	Fácil alcance radial y vertical.	La columna vertical rígida interfiere físicamente con piezas curvas adyacentes.
Antropomórfica (Seleccionada)	RRR	Excelente relación espacio de trabajo / huella base, flexibilidad para rodear el tubo y evitar colisiones.	Ecuaciones cinemáticas no lineales y acoplamiento dinámico.

La morfología antropomórfica RRR resulta idónea para esta aplicación al permitir que el brazo se “enrosque” o alcance por encima de la curvatura de la pieza cilíndrica sin que sus eslabones interfieran con ella, manteniendo una alta rigidez en un diseño compacto. Su espacio de trabajo esperado se caracteriza por una envoltura esférica truncada determinada por los límites de las articulaciones físicas del prototipo:

- Articulación 1 (Base): $\theta_1 \in [-90^\circ, 90^\circ]$ (debido a la restricción física de 180° del servomotor).
- Articulación 2 (Hombro): $\theta_2 \in [-90^\circ, 90^\circ]$ (restringido en firmware a $[-10^\circ, 90^\circ]$ por seguridad estructural).
- Articulación 3 (Codo): $\theta_3 \in [-90^\circ, 90^\circ]$ (restringido en firmware a $[-120^\circ, 10^\circ]$ para evitar colisión con el plano de soporte).
- Alcance radial máximo: $R_{\text{max}} = a_2 + a_3 + a_4 = 322,000$ mm

B. Sistemas de Referencia y Parámetros D-H

Se definen cuatro sistemas de referencia locales $\{0\}$, $\{1\}$, $\{2\}$ y $\{3\}$ asociados a cada articulación móvil y al efector final de la estructura, coincidiendo con los marcos de simulación.

- $d_1 = 60$ mm: Altura del pedestal de la base.
- $d_2 = 36$ mm: Altura del hombro.
- $a_2 = 15$ mm: Desfase radial base-hombro.
- $a_3 = 146$ mm: Longitud del primer eslabón móvil.
- $a_4 = 161$ mm: Longitud del segundo eslabón móvil.
- $d_4 = -7$ mm: Desplazamiento transversal del efector.
- θ_1 : Ángulo de rotación de la base (alrededor del eje vertical Z_0).
- θ_2 : Ángulo del hombro.
- θ_3 : Ángulo del codo.

La convención clásica de D-H establece que la matriz de transformación homogénea entre eslabones consecutivos se calcula mediante:

$$T_i^{i-1} = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (1)$$

A partir de los sistemas de referencia y dimensiones físicas asignados, se obtiene la Tabla de Parámetros D-H (Tabla II). La Fig. 1 ilustra la disposición geométrica de estos parámetros sobre el modelo físico del brazo en su configuración de reposo ($\theta_1 = \theta_2 = \theta_3 = 0$).

Tabla II
PARÁMETROS DE DENAVIT-HARTENBERG REALES DEL PROTOTIPO

Eslabón (i)	d_i (mm)	θ_i	a_i (mm)	α_i
1 ($0 \rightarrow 1$)	$d_1 + d_2 = 96$	θ_1	$a_2 = 15$	90°
2 ($1 \rightarrow 2$)	0	θ_2	$a_3 = 146$	0
3 ($2 \rightarrow 3$)	$d_4 = -7$	θ_3	$a_4 = 161$	0

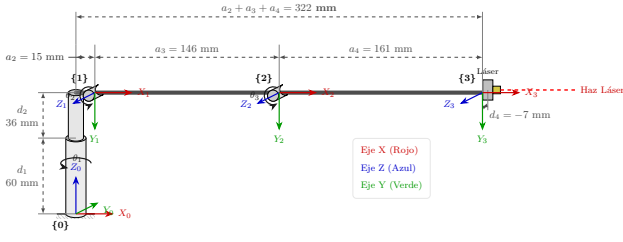


Figura 1. Disposición geométrica del brazo robótico láser en configuración de reposo ($\theta_1 = \theta_2 = \theta_3 = 0$). Se indican los sistemas de referencia D-H $\{0\}$ – $\{3\}$, las longitudes de eslabón a_3 , a_4 , los desplazamientos d_1 , d_2 y el desfase transversal d_4 .

B1. Mapeo de Variables Articulares y Desfases Geométricos: Es fundamental destacar que las variables articulares θ_i de la Tabla II corresponden al modelo teórico de Denavit-Hartenberg (donde el brazo totalmente extendido horizontalmente define los ángulos de hombro y codo $\theta_2 = \theta_3 = 0$). Sin embargo, en la construcción física del prototipo y en el modelo descriptor URDF de ROS 2/Gazebo, los servomotores se calibran en posiciones angulares relativas a sus ejes de montaje estructural. La correspondencia matemática exacta entre los ángulos teóricos de D-H (θ_2^* , θ_3^*) y los ángulos físicos comandados a los servomotores (θ_2 , θ_3) está definida por los desfases angulares mecánicos $\phi_2 \approx 59,017^\circ$ (1,030041 rad) y $\phi_3 \approx -10,966^\circ$ (-0,191392 rad):

$$\theta_2^* = \theta_2 + \phi_2 \quad (2)$$

$$\theta_3^* = \theta_3 + \phi_3 - \phi_2 \quad (3)$$

Donde θ_2^* y θ_3^* son los ángulos efectivos utilizados en las matrices de transformación homogénea y en el cálculo de cinemática directa analítica, mientras que θ_2 y θ_3 son los ángulos comandados desde el software de control a las articulaciones físicas del robot.

C. Cinemática Directa

Sustituyendo los parámetros de la Tabla II en (1), se obtienen las matrices de transformación homogénea individuales para cada articulación y eslabón:

$$T_1^0 = \begin{bmatrix} \cos \theta_1 & 0 & \sin \theta_1 & a_2 \cos \theta_1 \\ \sin \theta_1 & 0 & -\cos \theta_1 & a_2 \sin \theta_1 \\ 0 & 1 & 0 & d_1 + d_2 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (4)$$

$$T_2^1 = \begin{bmatrix} \cos \theta_2 & -\sin \theta_2 & 0 & a_3 \cos \theta_2 \\ \sin \theta_2 & \cos \theta_2 & 0 & a_3 \sin \theta_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (5)$$

$$T_3^2 = \begin{bmatrix} \cos \theta_3 & -\sin \theta_3 & 0 & a_4 \cos \theta_3 \\ \sin \theta_3 & \cos \theta_3 & 0 & a_4 \sin \theta_3 \\ 0 & 0 & 1 & d_4 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (6)$$

Multiplicando consecutivamente estas matrices se obtiene la matriz de transformación homogénea total del efector final respecto a la base del robot $T_3^0 = T_1^0 T_2^1 T_3^2$. Definimos esta matriz en función de las coordenadas del efector final $[x_E, y_E, z_E]^T$ dadas en (8)-(10):

$$T_3^0 = \begin{bmatrix} C_1 C_{23} & -C_1 S_{23} & S_1 & x_E \\ S_1 C_{23} & -S_1 S_{23} & -C_1 & y_E \\ S_{23} & C_{23} & 0 & z_E \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (7)$$

donde se utiliza la notación compacta $C_1 = \cos \theta_1$, $S_1 = \sin \theta_1$, $C_{23} = \cos(\theta_2 + \theta_3)$ y $S_{23} = \sin(\theta_2 + \theta_3)$.

El vector de posición del efector final $P_E = [x_E, y_E, z_E]^T$ está dado por:

$$x_E = \cos \theta_1 (a_2 + a_3 \cos \theta_2 + a_4 \cos(\theta_2 + \theta_3)) + d_4 \sin \theta_1 \quad (8)$$

$$y_E = \sin \theta_1 (a_2 + a_3 \cos \theta_2 + a_4 \cos(\theta_2 + \theta_3)) - d_4 \cos \theta_1 \quad (9)$$

$$z_E = d_1 + d_2 + a_3 \sin \theta_2 + a_4 \sin(\theta_2 + \theta_3) \quad (10)$$

Para verificar la consistencia del modelo matemático desarrollado, se evalúan las ecuaciones de cinemática directa bajo la configuración de reposo del prototipo (con todas las articulaciones en sus posiciones centrales $\theta_1 = 0$, $\theta_2 = 0$, $\theta_3 = 0$). En este estado, al sustituir los valores y las constantes físicas en (8)-(10), se obtiene:

$$\begin{aligned} x_E &= a_2 + a_3 \cos(0) + a_4 \cos(0) \\ &= 15 + 146 + 161 = 322 \text{ mm} \end{aligned} \quad (11)$$

$$y_E = -d_4 = 7 \text{ mm} \quad (12)$$

$$\begin{aligned} z_E &= d_1 + d_2 + a_3 \sin(0) + a_4 \sin(0) \\ &= 60 + 36 + 0 + 0 = 96 \text{ mm} \end{aligned} \quad (13)$$

Estos resultados analíticos coinciden exactamente con las coordenadas de la brida (efector final) obtenidas directamente del árbol de transformadas (TF) de la simulación virtual de ROS 2 ($X = 322$ mm, $Y = 7$ mm, $Z = 96$ mm). Esto valida la precisión del modelo cinemático de 4 sistemas propuesto para el guiado de trayectorias.

Para hallar la distancia total en 3D desde el origen del sistema de referencia $\{0\}$ hasta el efector final en esta posición de reposo, se descompone el cálculo geométrico en dos etapas mediante el teorema de Pitágoras: 1) Primero, se considera la proyección bidimensional sobre el plano longitudinal del brazo (vista lateral, Fig. 2), omitiendo temporalmente el desplazamiento transversal d_4 . En esta vista, el vector de posición proyectado tiene componentes $(x_{\text{plane}}, z_{\text{plane}})$, donde:

$$x_{\text{plane}} = a_2 + a_3 + a_4 = 15 + 146 + 161 = 322 \text{ mm} \quad (14)$$

$$z_{\text{plane}} = d_1 + d_2 = 60 + 36 = 96 \text{ mm} \quad (15)$$

La distancia proyectada D_{plane} actúa como la hipotenusa de este primer triángulo rectángulo en el plano vertical:

$$\begin{aligned} D_{\text{plane}} &= \sqrt{x_{\text{plane}}^2 + z_{\text{plane}}^2} \\ &= \sqrt{322^2 + 96^2} \\ &= \sqrt{112900} = 336 \text{ mm} \end{aligned} \quad (16)$$

Es de destacar que esta combinación de parámetros físicos produce un valor entero exacto de 336 mm para la hipotenusa del plano de movimiento longitudinal. 2) Segundo, para determinar la distancia tridimensional real, se toma D_{plane} como el primer cateto y el desfase lateral $d_4 = 7$ mm como el segundo cateto de un nuevo triángulo rectángulo en el espacio 3D. La hipotenusa resultante representa la distancia total del efector final al origen:

$$\begin{aligned} D_{\text{total}} &= \sqrt{D_{\text{plane}}^2 + d_4^2} \\ &= \sqrt{336^2 + 7^2} \\ &= \sqrt{112949} \approx 336,08 \text{ mm} \end{aligned} \quad (17)$$

Este método valida geoméricamente que las coordenadas tridimensionales calculadas mediante el formalismo D-H son consistentes con la morfología del hardware real.

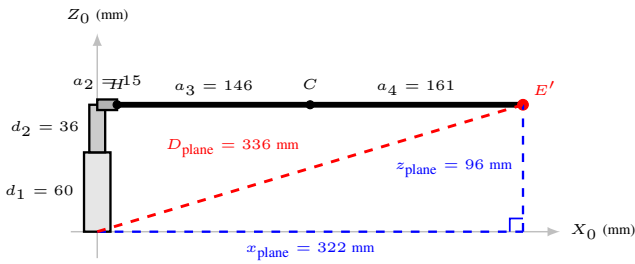


Figura 2. Deducción geométrica de la distancia proyectada en vista lateral (D_{plane}) en configuración de reposo ($\theta_1 = 0, \theta_2 = 0, \theta_3 = 0$). Los eslabones se alinean horizontalmente formando un triángulo rectángulo con catetos de 322 mm y 96 mm, resultando en una distancia proyectada exacta de 336 mm.

D. Cinemática Inversa y Análisis de Múltiples Soluciones

La resolución de la cinemática inversa consiste en determinar los ángulos físicos de los servomotores ($\theta_1, \theta_2, \theta_3$) a partir de una posición cartesiana deseada $P_E = [x_E, y_E, z_E]^T$. Para ello, desacoplamos el efecto de la rotación de la base y del desfase transversal d_4 de la siguiente manera:

1) *Cálculo de θ_1 (Rotación de la Base)*: Geométricamente, definimos el radio proyectado plano R_p como la longitud de la proyección horizontal de los eslabones principales del brazo en el suelo (desde el eje central de rotación de la base hasta la punta del brazo). A partir de la cinemática directa en (8)-(9), este radio está determinado por los eslabones físicos a_i y los ángulos del hombro y codo como:

$$R_p = a_2 + a_3 \cos \theta_2 + a_4 \cos(\theta_2 + \theta_3) \quad (18)$$

Para desacoplar el ángulo de la base θ_1 , consideramos la posición de la punta del brazo en el marco de referencia local del brazo $\{1\}$ (el cual gira con la base un ángulo θ_1). En dicho plano local, la coordenada a lo largo del brazo es $x_1 = R_p$ y la coordenada transversal debida al desfase transversal de la herramienta es $y_1 = -d_4$. La transformación de estas

coordenadas locales al marco global $\{0\}$ de la base se realiza mediante una matriz de rotación alrededor del eje vertical Z_0 :

$$\begin{bmatrix} x_E \\ y_E \end{bmatrix} = \begin{bmatrix} \cos \theta_1 & -\sin \theta_1 \\ \sin \theta_1 & \cos \theta_1 \end{bmatrix} \begin{bmatrix} R_p \\ -d_4 \end{bmatrix} \quad (19)$$

Al resolver el producto de la matriz anterior, obtenemos las ecuaciones de la cinemática directa:

$$x_E = R_p \cos \theta_1 + d_4 \sin \theta_1 \quad (20)$$

$$y_E = R_p \sin \theta_1 - d_4 \cos \theta_1 \quad (21)$$

Elevando al cuadrado y sumando ambas expresiones, el ángulo θ_1 se elimina mediante la identidad trigonométrica fundamental $\sin^2 \theta_1 + \cos^2 \theta_1 = 1$, lo que nos permite calcular directamente el valor numérico de R_p a partir de la coordenada cartesiana de consigna (x_E, y_E) :

$$x_E^2 + y_E^2 = R_p^2 + d_4^2 \implies R_p = \sqrt{x_E^2 + y_E^2 - d_4^2} \quad (22)$$

Realizando la sustitución de variables polares en el plano XY mediante $x_E = R \cos \beta$ y $y_E = R \sin \beta$ (donde $R = \sqrt{x_E^2 + y_E^2}$ y $\beta = \text{atan2}(y_E, x_E)$), las ecuaciones se transforman en:

$$R \cos(\beta - \theta_1) = R_p \quad (23)$$

$$R \sin(\beta - \theta_1) = -d_4 \quad (24)$$

Dividiendo ambas expresiones trigonométricas, obtenemos el ángulo diferencial respecto a la proyección cartesiana:

$$\beta - \theta_1 = \text{atan2}(-d_4, R_p) \quad (25)$$

Despejando θ_1 , se deriva la solución analítica exacta:

$$\theta_1 = \text{atan2}(y_E, x_E) - \text{atan2}(-d_4, R_p) \quad (26)$$

2) *Cálculo de θ_3 (Elevación del Codo)*: Se define la proyección en el plano longitudinal del brazo. El radio proyectado corregido r (distancia horizontal desde el eje del hombro al efector) y la altura corregida respecto al hombro z' se formulan como:

$$r = R_p - a_2 \quad (27)$$

$$z' = z_E - (d_1 + d_2) \quad (28)$$

Aplicando la ley de los cosenos en el triángulo formado por los eslabones a_3 y a_4 en el plano del brazo para resolver el ángulo del codo θ_3 :

$$r^2 + z'^2 = a_3^2 + a_4^2 - 2a_3a_4 \cos(180^\circ - \theta_3) \quad (29)$$

Considerando que $\cos(180^\circ - \theta_3) = -\cos \theta_3$, se tiene:

$$r^2 + z'^2 = a_3^2 + a_4^2 + 2a_3a_4 \cos \theta_3 \quad (30)$$

Despejando el término del coseno se obtiene:

$$\cos \theta_3 = \frac{r^2 + z'^2 - a_3^2 - a_4^2}{2a_3a_4} = \Psi \quad (31)$$

Para la existencia de una solución física real libre de números complejos, se debe cumplir estrictamente $-1 \leq \Psi \leq 1$. Físicamente, un valor fuera de este rango denota que la coordenada cartesiana solicitada se encuentra fuera del alcance

del manipulador (ya sea por estar muy lejana o excesivamente cercana a la base). En lugar de calcular el ángulo directamente mediante la función de arco coseno ($\theta_3 = \arccos \Psi$), la cual presenta inestabilidad numérica en las fronteras ($\Psi \approx \pm 1$) e incertidumbre respecto al signo del ángulo, se recurre al cálculo del seno de θ_3 mediante la identidad trigonométrica fundamental, $\sin \theta_3 = \pm \sqrt{1 - \Psi^2}$. Esto permite resolver el ángulo de forma numéricamente robusta en todo el rango $[-\pi, \pi]$ mediante el operador arcotangente de dos argumentos:

$$\theta_3 = \text{atan2}\left(\pm \sqrt{1 - \Psi^2}, \Psi\right) \quad (32)$$

donde el signo positivo (+) representa la configuración de codo abajo (Elbow Down) y el signo negativo (-) representa la de codo arriba (Elbow Up).

En el firmware del microcontrolador ESP32 y en la interfaz gráfica, se selecciona de forma exclusiva la configuración de codo arriba (utilizando el signo negativo). Esta decisión de diseño mecánico y de control evita colisiones físicas del eslabón distal con la mesa de trabajo o la base de soporte, asegurando un despeje máximo de la herramienta y mejorando la operabilidad en superficies no planas.

3) *Cálculo de θ_2 (Elevación del Hombro)*: Una vez obtenido θ_3 , se descompone la posición cartesiana en el plano del brazo:

$$r = a_3 \cos \theta_2 + a_4 \cos(\theta_2 + \theta_3) \quad (33)$$

$$z' = a_3 \sin \theta_2 + a_4 \sin(\theta_2 + \theta_3) \quad (34)$$

Expandiendo las identidades trigonométricas de la suma de ángulos y agrupando los términos:

$$r = (a_3 + a_4 \cos \theta_3) \cos \theta_2 - (a_4 \sin \theta_3) \sin \theta_2 \quad (35)$$

$$z' = (a_3 + a_4 \cos \theta_3) \sin \theta_2 + (a_4 \sin \theta_3) \cos \theta_2 \quad (36)$$

Definiendo las constantes auxiliares sintonizadas $k_1 = a_3 + a_4 \cos \theta_3$ y $k_2 = a_4 \sin \theta_3$, se reescribe el sistema lineal como:

$$r = k_1 \cos \theta_2 - k_2 \sin \theta_2 \quad (37)$$

$$z' = k_1 \sin \theta_2 + k_2 \cos \theta_2 \quad (38)$$

Resolviendo el sistema algebraico para los términos de $\sin \theta_2$ y $\cos \theta_2$:

$$\sin \theta_2 = \frac{k_1 z' - k_2 r}{k_1^2 + k_2^2} \quad (39)$$

$$\cos \theta_2 = \frac{k_1 r + k_2 z'}{k_1^2 + k_2^2} \quad (40)$$

Dividiendo ambas expresiones se determina de manera única el ángulo de elevación del hombro:

$$\theta_2 = \text{atan2}(k_1 z' - k_2 r, k_1 r + k_2 z') \quad (41)$$

Alternativamente, aplicando el método geométrico directo para un manipulador planar de 2 GDL (con eslabones a_3 y a_4), el ángulo del hombro θ_2 se formula a partir de la diferencia de ángulos en el triángulo del brazo:

$$\theta_2 = \text{atan2}(z', r) - \text{atan2}(a_4 \sin \theta_3, a_3 + a_4 \cos \theta_3) \quad (42)$$

La equivalencia matemática entre la solución algebraica (41) y la solución geométrica (42) se demuestra mediante la

identidad de la diferencia de arcotangentes, $\text{atan2}(A, B) - \text{atan2}(C, D) = \text{atan2}(AD - BC, BD + AC)$, lo que valida la coherencia cinemática del modelo embebido en el microcontrolador.

D1. Gestión de Límites Articulares y Puntos Inalcanzables: Para garantizar la integridad física del manipulador y evitar fallas críticas de ejecución en el resolutor matemático:

- **Detección de Coordenadas Fuera de Alcance**: El algoritmo evalúa el valor de Ψ en (31) antes de calcular la raíz cuadrada en (32). Si $|\Psi| > 1,0$, la coordenada solicitada se encuentra geoméricamente fuera de la frontera de alcance del robot (muy lejana o muy cercana a la base). En este caso, el software descarta la consigna cartesiana, genera una alerta visual de tipo OUT_OF_RANGE y detiene el movimiento del láser. Se aplica una pequeña tolerancia de $\pm 0,005$ para compensar errores de truncamiento decimal de punto flotante en las fronteras extremas del espacio de trabajo.
- **Filtro de Límites Articulares**: Una vez calculada la terna $(\theta_1, \theta_2, \theta_3)$, se comprueba que se cumplan las restricciones físicas de giro de los actuadores del prototipo real (limitados a un rango de operación física de 180°):

$$-90^\circ \leq \theta_1 \leq 90^\circ \quad (43)$$

$$-90^\circ \leq \theta_2 \leq 90^\circ \quad (44)$$

$$-90^\circ \leq \theta_3 \leq 90^\circ \quad (45)$$

Adicionalmente, el firmware del ESP32 restringe de forma más estricta estos márgenes angulares para garantizar la seguridad del hardware frente a sobrecorrientes o colisiones con la mesa de trabajo:

$$-10^\circ \leq \theta_2 \leq 90^\circ \quad (46)$$

$$-120^\circ \leq \theta_3 \leq 10^\circ \quad (47)$$

Cualquier consigna que viole estos límites se descarta automáticamente para evitar colisiones mecánicas directas o sobre-esfuerzo de los servomotores.

E. Jacobiano, Singularidades y Caracterización del Espacio de Trabajo

La cinemática diferencial del manipulador relaciona las velocidades del espacio articular con las velocidades lineales del efector en el espacio cartesiano. Para obtener la matriz Jacobiana $J_v \in \mathbb{R}^{3 \times 3}$, se recurre al método geométrico (productos vectoriales basados en los sistemas de referencia D-H), siendo el más conveniente por su simplicidad en articulaciones rotacionales, expresándose columna por columna como:

$$J_{v_i} = z_{i-1} \times (p_e - p_{i-1}) \quad (48)$$

Al aplicar esta metodología a nuestro manipulador de 3 GDL, se definen los ejes de rotación en el sistema de referencia global:

$$z_0 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \quad z_1 = z_2 = \begin{bmatrix} \sin \theta_1 \\ -\cos \theta_1 \\ 0 \end{bmatrix} \quad (49)$$

y los vectores de posición relativos del efector p_e respecto a la base p_0 y al codo p_2 :

$$p_e - p_0 = \begin{bmatrix} R_p \cos \theta_1 + d_4 \sin \theta_1 \\ R_p \sin \theta_1 - d_4 \cos \theta_1 \\ z_e \end{bmatrix} \quad (50)$$

$$p_e - p_2 = \begin{bmatrix} a_4 \cos(\theta_2 + \theta_3) \cos \theta_1 + d_4 \sin \theta_1 \\ a_4 \cos(\theta_2 + \theta_3) \sin \theta_1 - d_4 \cos \theta_1 \\ a_4 \sin(\theta_2 + \theta_3) \end{bmatrix} \quad (51)$$

donde $R_p = a_2 + a_3 \cos \theta_2 + a_4 \cos(\theta_2 + \theta_3)$ es el radio proyectado del brazo. Al calcular los productos vectoriales cruzados, se obtienen las columnas de la velocidad lineal:

$$J_{v1} = z_0 \times (p_e - p_0) = \begin{bmatrix} -R_p \sin \theta_1 + d_4 \cos \theta_1 \\ R_p \cos \theta_1 + d_4 \sin \theta_1 \\ 0 \end{bmatrix} \quad (52)$$

$$J_{v3} = z_2 \times (p_e - p_2) = \begin{bmatrix} -a_4 \cos \theta_1 \sin(\theta_2 + \theta_3) \\ -a_4 \sin \theta_1 \sin(\theta_2 + \theta_3) \\ a_4 \cos(\theta_2 + \theta_3) \end{bmatrix} \quad (53)$$

Al agrupar las tres columnas del manipulador, se conforma la matriz Jacobiana lineal final:

$$J_v = \begin{bmatrix} -S_1 R_p + d_4 C_1 & -C_1(a_3 S_2 + a_4 S_{23}) & -C_1 a_4 S_{23} \\ C_1 R_p + d_4 S_1 & -S_1(a_3 S_2 + a_4 S_{23}) & -S_1 a_4 S_{23} \\ 0 & a_3 C_2 + a_4 C_{23} & a_4 C_{23} \end{bmatrix} \quad (54)$$

donde se emplea la notación compacta S_i , C_i , S_{23} y C_{23} para representar los senos y cosenos directos. Esto demuestra de forma analítica que el método geométrico entrega exactamente el mismo resultado que la diferenciación analítica.

E1. Cálculo Simplificado del Determinante: El determinante de la matriz Jacobiana J_v es una herramienta analítica clave para determinar las singularidades cinemáticas del robot. Para calcularlo de forma simplificada, se realiza la expansión por cofactores a lo largo de la tercera fila de J_v en (54). Dado que esta fila posee un elemento nulo ($J_{31} = 0$), el determinante se reduce a la suma de dos términos:

$$\det(J_v) = -J_{32}(J_{11}J_{23} - J_{13}J_{21}) + J_{33}(J_{11}J_{22} - J_{12}J_{21}) \quad (55)$$

Este cálculo se puede simplificar sustancialmente mediante tres pasos clave:

1. **Eliminación de la orientación de la base (θ_1):** Al expandir los productos cruzados dentro de los paréntesis de (55), se factoriza de forma natural el término $\sin^2 \theta_1 + \cos^2 \theta_1$ (utilizando la notación compacta $S_1^2 + C_1^2$). Por la identidad pitagórica fundamental, este término es igual a 1, eliminando por completo la influencia de θ_1 :

$$J_{11}J_{23} - J_{13}J_{21} = a_4 S_{23} R_p \quad (56)$$

$$J_{11}J_{22} - J_{12}J_{21} = (a_3 S_2 + a_4 S_{23}) R_p \quad (57)$$

Esto demuestra analíticamente que la orientación angular de la base no afecta las singularidades de posicionamiento del brazo.

2. **Agrupamiento de términos en el plano:** Al sustituir las simplificaciones anteriores y los coeficientes de la

tercera fila $J_{32} = a_3 C_2 + a_4 C_{23}$ y $J_{33} = a_4 C_{23}$ en (55), se cancelan los términos que contienen a_4^2 , reduciendo la expresión a:

$$\det(J_v) = a_3 a_4 R_p (S_2 C_{23} - C_2 S_{23}) \quad (58)$$

o de forma equivalente en términos trigonométricos expandidos:

$$\det(J_v) = a_3 a_4 R_p (\sin \theta_2 \cos(\theta_2 + \theta_3) - \cos \theta_2 \sin(\theta_2 + \theta_3)) \quad (59)$$

3. Identidad del seno de la diferencia de ángulos:

Utilizando la conocida identidad trigonométrica $\sin(A - B) = \sin A \cos B - \cos A \sin B$ (donde definimos $A = \theta_2$ y $B = \theta_2 + \theta_3$, de modo que $A - B = -\theta_3$ y $\sin(-\theta_3) = -\sin \theta_3$), se llega de forma directa a la formulación final compacta del determinante:

$$\det(J_v) = -a_3 a_4 R_p \sin \theta_3 \quad (60)$$

Esta expresión demuestra que el determinante es independiente del desfase transversal d_4 , lo que significa que el descentramiento lateral de la herramienta no influye en las condiciones de bloqueo del brazo.

E2. Cinemática Diferencial Inversa y Aplicación Numérica: Para controlar la velocidad de avance del cabezal láser a lo largo del contorno y mantener un haz de corte homogéneo, se calcula la cinemática diferencial inversa, la cual relaciona la velocidad cartesiana lineal de consigna $\dot{P} = [\dot{x}, \dot{y}, \dot{z}]^T$ con las velocidades angulares requeridas en las articulaciones $\dot{q} = [\dot{\theta}_1, \dot{\theta}_2, \dot{\theta}_3]^T$ a través de $\dot{q} = J_v^{-1} \dot{P}$.

Debido a que el determinante de J_v se dedujo en (60), es posible invertir la matriz de forma analítica desacoplando el movimiento en el plano radial y transversal. Al proyectar la velocidad cartesiana sobre el marco giratorio de la base, la velocidad angular de la base y la tasa de cambio del radio proyectado se expresan como:

$$\dot{\theta}_1 = \frac{-\dot{x} \sin \theta_1 + \dot{y} \cos \theta_1}{R_p} \quad (61)$$

$$\dot{R}_p = \dot{x} \cos \theta_1 + \dot{y} \sin \theta_1 - d_4 \dot{\theta}_1 \quad (62)$$

Con el valor de $\dot{R}_p$ y la velocidad vertical deseada $\dot{z}$, se resuelve el sistema coplanar para el hombro y el codo empleando la inversa de la submatriz planar J_{planar} :

$$\dot{\theta}_2 = \frac{a'_4 C_{23} \dot{R}_p + a'_4 S_{23} \dot{z}}{a_3 a'_4 S_3} \quad (63)$$

$$\dot{\theta}_3 = \frac{-(a_3 C_2 + a'_4 C_{23}) \dot{R}_p - (a_3 S_2 + a'_4 S_{23}) \dot{z}}{a_3 a'_4 S_3} \quad (64)$$

donde se emplea la notación compacta $S_3 = \sin \theta_3$, $C_2 = \cos \theta_2$, $S_2 = \sin \theta_2$, $C_{23} = \cos(\theta_2 + \theta_3)$ y $S_{23} = \sin(\theta_2 + \theta_3)$.

A modo de verificación práctica de estas ecuaciones de velocidad, se evalúa numéricamente el comportamiento dinámico del manipulador real con el eslabón extendido del láser $a'_4 = 361$ mm en la configuración articular $\theta_1 = 0$, $\theta_2 = 30^\circ$ (0,5236 rad) y $\theta_3 = -45^\circ$ (-0,7854 rad), asumiendo un requerimiento de velocidad lineal pura en sentido transversal

(eje Y) de $\dot{P} = [0, 100, 0]^T$ mm/s (equivalente a un barrido transversal de 10 cm/s sobre la pieza):

1) *Cálculo del radio proyectado R_p :*

$$\begin{aligned} R_p &= a_2 + a_3 \cos \theta_2 + a_4' \cos(\theta_2 + \theta_3) \\ &= 15 + 146 \cos(30^\circ) + 361 \cos(-15^\circ) \\ &\approx 15 + 126,44 + 348,69 = 490,13 \text{ mm} \end{aligned}$$

2) *Determinación de las velocidades articulares:*

■ Sustituyendo en (61) con $\sin(0) = 0$ y $\cos(0) = 1$:

$$\dot{\theta}_1 = \frac{-0 \cdot 0 + 100 \cdot 1}{490,13} \approx 0,2040 \text{ rad/s} \quad (\approx 11,69^\circ/\text{s})$$

■ A partir del valor de $\dot{\theta}_1$, calculamos $\dot{R}_p$ mediante (62):

$$\dot{R}_p = 0 \cdot 1 + 100 \cdot 0 - (-7) \cdot 0,2040 \approx 1,428 \text{ mm/s}$$

■ Con $\dot{R}_p = 1,428$ mm/s y $\dot{z} = 0$, resolvemos las velocidades angulares del hombro y del codo a través de (63) y (64):

$$\begin{aligned} \dot{\theta}_2 &= \frac{361 \cos(-15^\circ) \cdot 1,428 + 0}{146 \cdot 361 \sin(-45^\circ)} \\ &= \frac{497,93}{-37267,4} \approx -0,0134 \text{ rad/s} \quad (\approx -0,77^\circ/\text{s}) \\ \dot{\theta}_3 &= \frac{-(146 \cos(30^\circ) + 361 \cos(-15^\circ)) \cdot 1,428 - 0}{146 \cdot 361 \sin(-45^\circ)} \\ &= \frac{-678,53}{-37267,4} \approx 0,0182 \text{ rad/s} \quad (\approx 1,04^\circ/\text{s}) \end{aligned}$$

Este cálculo demuestra cómo se distribuye vectorialmente la velocidad de consigna cartesiana en las juntas mecánicas, permitiendo que el software en la computadora host controle la dinámica del movimiento sin saturar el torque de los servomotores físicos.

E3. Análisis Físico de Singularidades: Las singularidades cinemáticas son configuraciones o "posturas" específicas donde el robot se bloquea y pierde la capacidad de moverse en alguna dirección del espacio. Matemáticamente ocurren cuando $\det(J_v) = 0$, lo que provoca que el sistema exija velocidades infinitas en los servomotores para realizar movimientos cartesianos mínimos, generando sobrecorrientes y fallos. En este manipulador se identifican dos singularidades críticas:

- **Singularidad del Codo** ($\sin \theta_3 = 0 \implies \theta_3 = 0$ ó $\theta_3 = -\pi$): Ocurre cuando el brazo está completamente estirado o totalmente encogido. En esta posición, el efector no puede realizar ningún movimiento en la dirección radial del brazo (no puede estirarse ni encogerse más). Si se le ordena avanzar en esa dirección, los servomotores del hombro y del codo tendrían que moverse a una velocidad teóricamente infinita para obedecer la consigna, lo que en la práctica produce un bloqueo mecánico del brazo, picos extremos de corriente eléctrica que pueden dañar los motores y el reinicio del microcontrolador ESP32 debido a la sobrecarga.
- **Singularidad de la Base** ($R_p = 0$): Ocurre cuando el codo se repliega de tal manera que el efector final queda alineado exactamente sobre el eje vertical de rotación de

la base (Z_0). En esta pose, si se le ordena al láser un movimiento horizontal perpendicular al brazo, la base tendría que girar instantáneamente un ángulo de 90° (velocidad de giro teóricamente infinita, $\dot{\theta}_1 \rightarrow \infty$). Esto genera fuertes oscilaciones, vibraciones inestables y la pérdida del control de posicionamiento en el prototipo físico.

E4. Caracterización y Análisis Gráfico del Espacio de Trabajo: El espacio de trabajo útil del brazo manipulador antropomórfico está delimitado geométricamente por los alcances mínimos y máximos de la cadena cinemática, truncados por las restricciones angulares de los servomotores. La Fig. 3 representa la sección transversal de la envolvente de trabajo en el plano vertical (R_p -Z).

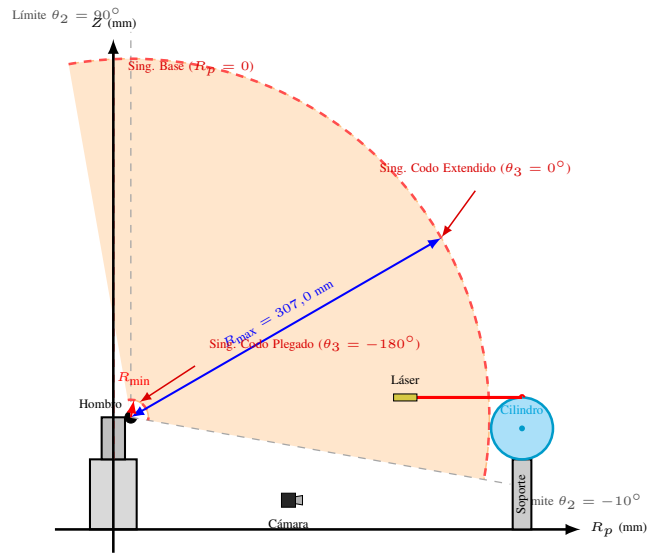


Figura 3. Espacio de trabajo (sección transversal R_p -Z) con eslabones en posición de enfoque y límites de $\theta_2 \in [-10^\circ, 90^\circ]$. Se destacan en líneas discontinuas rojas las zonas de singularidad del Jacobiano: base ($R_p = 0$) y codo (fronteras $R_{\min}$ y $R_{\max}$).

F. Generación de Trayectorias y Planificación en el Espacio Cartesiano

Para garantizar la calidad y precisión del corte láser, se implementa una planificación en el espacio cartesiano. La trayectoria requerida sobre la pieza cilíndrica se discretiza en una secuencia ordenada de puntos tridimensionales. Para cada punto de esta secuencia, la computadora host calcula la solución exacta del modelo de cinemática inversa y transmite los ángulos de articulación resultantes al microcontrolador ESP32 a una frecuencia constante de 20 Hz.

El algoritmo de planificación proyecta cada píxel del dibujo 2D (p_x, p_y) en una imagen de dimensiones $W_u \times H_u$ a una coordenada cartesiana tridimensional sobre la superficie del

cilindro (x_s, y_s, z_s) usando el mapeo inverso:

$$x_s = X_c - R_c \cos \theta \quad (65)$$

$$y_s = Y_c - \left(\frac{p_x}{W_u - 1} - 0,5 \right) \cdot W_{\text{flat}} \quad (66)$$

$$z_s = Z_c + R_c \sin \theta \quad (67)$$

donde $X_c = 500$ mm, $Y_c = 0$ mm, $Z_c = 55$ mm es la posición central del cilindro, $R_c = 15$ mm es su radio, $W_{\text{flat}} = 60$ mm es el ancho plano extendido y $\theta = \theta_{\text{max}} - \frac{p_y}{H_u - 1} \cdot 2\theta_{\text{max}}$ (con $\theta_{\text{max}} = 1,25$ rad) representa la rotación angular del envoltorio.

Para que el haz láser corte de manera efectiva, el efector final físico (punto de salida del haz en la brida) debe situarse a una distancia focal constante de $d_f = 200$ mm respecto a la superficie del cilindro. Para implementar esta restricción sin agregar complejidad geométrica al resolutor de cinemática inversa, se aplica una compensación virtual del largo del eslabón terminal en el resolutor IK. En lugar de resolver la cinemática inversa con la longitud física del tercer eslabón $a_4 = 161$ mm, el resolutor utiliza un eslabón ficticio extendido:

$$a'_4 = a_4 + d_f = 161 + 200 = 361 \text{ mm} \quad (68)$$

Al ingresar las coordenadas de superficie (x_s, y_s, z_s) como objetivo en la cinemática inversa usando a'_4 , la junta del codo se posiciona de tal forma que la brida física se sitúa exactamente a 200 mm normales de la superficie del cilindro, manteniendo constante el enfoque del láser durante todo el movimiento.

A modo de ejemplo numérico de aplicación del algoritmo de planificación de trayectorias, se realiza el cálculo completo para un punto de consigna ubicado en el centro de la imagen de entrada ($p_x = 100$, $p_y = 100$) en una cuadrícula de 200×200 píxeles ($W_u = 200$, $H_u = 200$):

1) Cálculo de la coordenada angular θ sobre el cilindro:

$$\theta = 1,25 - \frac{100}{199} \cdot 2,50 \approx -0,0063 \text{ rad}$$

2) Mapeo a coordenadas cartesianas 3D sobre la superficie mediante (65)-(67):

$$x_s = 500 - 15 \cos(-0,0063) \approx 485,00 \text{ mm}$$

$$y_s = 0 - \left(\frac{100}{199} - 0,5 \right) \cdot 60 \approx -0,15 \text{ mm}$$

$$z_s = 55 + 15 \sin(-0,0063) \approx 54,91 \text{ mm}$$

3) Compensación focal y resolución de la cinemática inversa:

• **Radio proyectado en la base R_p :** Usando el eslabón extendido $a'_4 = 361$ mm, calculamos el radio proyectado R_p :

$$\begin{aligned} r_s &= \sqrt{x_s^2 + y_s^2} \\ &= \sqrt{485,00^2 + (-0,15)^2} \approx 485,00 \text{ mm} \\ R_p &= \sqrt{r_s^2 - d_4^2} \\ &= \sqrt{485,00^2 - (-7)^2} \approx 484,95 \text{ mm} \end{aligned}$$

• **Ángulo de la articulación de la base θ_1 :**

$$\begin{aligned} \theta_1 &= \text{atan2}(-0,15, 485) - \text{atan2}(7, 484,95) \\ &\approx -0,0147 \text{ rad} \quad (\approx -0,84^\circ) \end{aligned}$$

• **Variables coplanares corregidas respecto al hombro:**

$$x_c = R_p - a_2 = 484,95 - 15 = 469,95 \text{ mm}$$

$$z_c = z_s - (d_1 + d_2) = 54,91 - 96 = -41,09 \text{ mm}$$

• **Ángulo de la articulación del codo θ_3 (configuración codo-arriba):**

$$\Psi = \frac{469,95^2 + (-41,09)^2 - 146^2 - 361^2}{2 \cdot 146 \cdot 361}$$

$$\approx 0,6724$$

$$\theta_3 = \text{atan2}\left(-\sqrt{1 - 0,6724^2}, 0,6724\right)$$

$$\approx -0,8335 \text{ rad} \quad (\approx -47,76^\circ)$$

• **Ángulo de la articulación del hombro θ_2 :**

$$k_1 = a_3 + a'_4 \cos \theta_3$$

$$= 146 + 361(0,6724) \approx 388,74 \text{ mm}$$

$$k_2 = a'_4 \sin \theta_3$$

$$= 361 \sin(-0,8335) \approx -267,22 \text{ mm}$$

$$\theta_2 = \text{atan2}(k_1 z_c - k_2 x_c, k_1 x_c + k_2 z_c)$$

$$\approx 0,5147 \text{ rad} \quad (\approx 29,49^\circ)$$

El resolutor cartesiano ejecuta estas operaciones para cada punto de la trayectoria en la laptop de forma instantánea y sin experimentar ningún retraso en los cálculos, transmitiendo de manera determinista los ángulos articulares al microcontrolador mediante conexión serie.

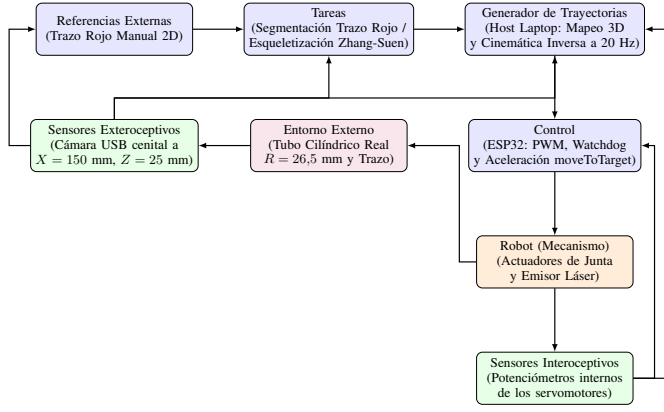


Figura 4. Esquema funcional de lazo de control del brazo robótico de 3 GDL para corte y grabado láser no-coplanar en formato de columna simple (media página).

IV. ARQUITECTURA Y ESQUEMA FUNCIONAL DEL ROBOT

Para comprender de manera integral la operación y el guiado automático del manipulador robótico de 3 GDL, se estructuró una arquitectura de control en lazo cerrado basada en el flujo de información entre las consignas de usuario, los resolvers matemáticos del host, el controlador físico de bajo nivel y los sistemas de retroalimentación sensorial. En la Fig. 4 se detalla el esquema funcional propuesto, el cual adapta el modelo de control en lazo cerrado clásico con sensores exteroceptivos e interoceptivos.

A continuación, se detalla el funcionamiento de cada uno de los bloques y su integración en el sistema físico real:

A. Referencias Externas (Trazo Rojo Manual 2D)

Este bloque representa la consigna de entrada o la trayectoria de referencia deseada en el espacio de la tarea. En el sistema desarrollado, si bien en el entorno de simulación virtual se cargaban letras o vectores predefinidos, en la realidad la referencia consiste en un trazo continuo dibujado a mano alzada con un rotulador de color **Rojo** directamente sobre la superficie cilíndrica de trabajo. El brazo robótico debe detectar y seguir físicamente este contorno dibujado, permitiendo una interacción directa y guiada por el trazo real en la pieza física.

B. Tareas (Segmentación Trazo Rojo y Esqueletización Zhang-Suen)

Es la primera etapa de procesamiento de software ejecutada en la computadora host (laptop). Su objetivo es procesar la información de imagen capturada por la cámara USB cenital y extraer de forma determinista la trayectoria geométrica ordenada del trazo físico. Inicialmente, el sistema realiza una mascarilla cian sobre la imagen para aislar y modelar la posición tridimensional del cilindro físico (cuyo cuerpo es de color cian). Posteriormente, para la tarea de guiado de corte, el software segmenta específicamente el **Trazo Rojo** manual aplicando una máscara de color en el espacio HSV

con rangos duales de tono (para cubrir la transición de rojo en los extremos $0^\circ-10^\circ$ y $165^\circ-180^\circ$).

Sobre esta máscara binaria limpia del trazo rojo, se ejecuta el algoritmo de esqueletización de Zhang-Suen para reducir las líneas a un espesor uniforme de un único píxel. Finalmente, la trayectoria se ordena utilizando el algoritmo del vecino más cercano y se simplifica mediante aproximación poligonal RDP para evitar movimientos discretos bruscos en los actuadores.

C. Generador de Trayectorias (Mapeo 3D y Cinemática Inversa)

Constituye el resolver cinemático y de trayectoria principal, operando en tiempo real en la computadora host a una frecuencia constante de 20 Hz. Este bloque recibe la ruta de píxeles secuenciada y realiza las siguientes operaciones consecutivas en la CPU de la laptop:

1. Proyección espacial sobre el cilindro mediante las ecuaciones de mapeo inverso (65)-(67), transformando las coordenadas de píxeles (p_x, p_y) a coordenadas cartesianas tridimensionales reales (x_s, y_s, z_s) en milímetros sobre la superficie curva.
2. Aplicación de la compensación de la distancia de enfoque, la cual utiliza el eslabón extendido virtual $a'_4 = a_4 + d_f = 361$ mm de (68) en lugar de la longitud física de la brida ($a_4 = 161$ mm), forzando mecánicamente al cabezal del láser a permanecer a una distancia focal constante de 200 mm perpendicular a la superficie cilíndrica.
3. Resolución del resolver cinemático inverso analítico codo-arriba para cada punto de la trayectoria 3D, produciendo la secuencia de ángulos articulares $(\theta_1, \theta_2, \theta_3)$ que se transmiten de manera determinista al hardware a través de la conexión serial.

El resolver cartesiano ejecuta estas operaciones en la CPU del host de forma síncrona, eliminando los retrasos de cálculo en el lazo.

D. Control (Microcontrolador ESP32)

Este bloque representa el controlador digital de bajo nivel de hardware embebido en el microcontrolador físico ESP32. Su función principal es recibir las posiciones articulares $(\theta_1, \theta_2, \theta_3)$ en radianes provenientes del host, validar que cumplan estrictamente con las restricciones de límites angulares físicos para evitar colisiones estructurales, y convertirlas a anchos de pulso de microsegundos de la señal PWM para accionar los servomotores físicos. La recepción se implementa a través de la interfaz UART a 115200 bps mediante tramas de texto estructuradas.

Adicionalmente, el controlador ejecuta tres rutinas críticas en su bucle periódico:

- **Aceleración moveToTarget():** Un algoritmo de interpolación lineal síncrono que divide los cambios angulares repentinos en incrementos máximos de $15 \mu s$ ($\approx 1,35^\circ$) espaciados por retardos temporales fijos de 15 ms, eliminando los picos de corriente instantáneos en la fuente

de alimentación externa que provocan caídas de tensión y el apagado del microcontrolador.

- **Conmutación Síncrona del Láser:** Gestiona de manera coordinada el pin digital GPIO 25 para encender y apagar el haz de luz coherente de forma sincronizada con el perfil cinemático de movimiento, previniendo marcas quemadas accidentales en fases de reposo.
- **Watchdog de Seguridad:** Un temporizador por software que comprueba la llegada periódica de comandos seriales. Si transcurren más de 1000 ms sin recibir comandos válidos del host, el microcontrolador desactiva inmediatamente el láser digital para prevenir quemaduras incidentales o incendios en la pieza de trabajo por pérdida de conexión.

E. Robot (Mecanismo)

Consiste en la planta física y mecánica del brazo manipulador antropomórfico de 3 GDL. Está estructurado por la base giratoria y los eslabones articulados fabricados en impresión 3D FDM utilizando plástico PETG de alta rigidez, los tres servomotores que actúan sobre las juntas activas (un MG996R en la base, un MG996R en el hombro y un MG90S en el codo) y el emisor de diodo láser montado en la brida final. Este bloque ejecuta mecánicamente los comandos angulares de junta transformando las señales eléctricas de PWM en torques de eje que desplazan angularmente los eslabones, posicionando espacialmente el emisor y proyectando el haz de luz sobre el entorno cilíndrico.

F. Entorno Externo (Pieza Cilíndrica de Trabajo)

Es el plano y volumen físico sobre el cual interacciona la herramienta final del robot. En este sistema de estudio, el entorno consiste en un tubo cilíndrico de radio real $R_{\text{tubo}} = 26,5$ mm (diámetro 5,3 cm) montado sobre un soporte horizontal con una altura de base de 60 mm y situado a una distancia radial fija de $x_{\text{tubo}} = 350$ mm con respecto al origen de la base del brazo. La superficie del tubo cilíndrico es la que recibe la proyección de la trayectoria y experimenta la alteración física (grabado o corte térmico) provocada por la colisión del haz de luz láser enfocado. Su geometría introduce una variación no lineal de altura en función del radio de giro.

G. Sensores Exteroceptivos (Cámara USB Cenital)

Este bloque de sensores proporciona el lazo de realimentación visual externo. Una cámara USB física externa está montada de forma rígida a una distancia fija de $X = 150$ mm y una altura de $Z = 25$ mm con respecto a la base del robot. La cámara captura continuamente la escena física (el cilindro y el trazo dibujado) y transmite la señal de video (RGB de 640x480 píxeles a 30 fps) a la laptop host. El software utiliza este feed de video óptico cenital para cerrar el lazo, permitiendo la auto-detección del color cian de la pieza cilíndrica, el cálculo de la homografía y la proyección inversa de dibujo plano sobre el contorno físico curvo en tiempo real, compensando cualquier error de alineación estructural.

H. Sensores Interoceptivos (Potenciómetros de Servomotores)

Constituyen el lazo de realimentación interno de posición de las articulaciones del manipulador. Cada servomotor físico (MG996R y MG90S) cuenta en su interior con un potenciómetro analógico de precisión montado solidariamente al eje de salida de los engranajes reductores y un controlador de control local analógico integrado. El potenciómetro lee constantemente el ángulo de giro real de la junta mecánica y el circuito comparador del servo ajusta instantáneamente la tensión del motor de corriente continua para anular la diferencia respecto a la señal de referencia PWM comandada por el ESP32. Esta realimentación ocurre internamente en cada servomotor a una frecuencia de aproximadamente 1 kHz, garantizando un posicionamiento articular preciso frente a torques de gravedad y fuerzas dinámicas externas sin sobrecargar el procesador del ESP32.

V. METODOLOGÍA Y ALCANCE

La ejecución del proyecto se estructuró en dos fases metodológicas consecutivas:

A. Fase 1: Modelado Mecánico y Simulación Virtual

Se diseñaron las piezas tridimensionales del manipulador en CAD. Los modelos tridimensionales de los eslabones se exportaron en formato STL para el proceso de manufactura, y se compilaron en un archivo descriptor URDF (con formato de mallas colisionables e inerciales). En el entorno de ROS 2 y Gazebo, se programó un nodo de simulación independiente para validar computacionalmente los modelos de cinemática directa e inversa, garantizando que el seguimiento de las trayectorias cartesianas de corte ocurra libre de colisiones mecánicas directas o singularidades. Esta simulación en ROS 2 y Gazebo se mantiene de forma puramente virtual, sin establecer conexiones directas de comunicación o control hacia el prototipo físico.

B. Fase 2: Fabricación e Integración de Hardware

La estructura mecánica se fabricó mediante impresión 3D FDM de los archivos STL utilizando plástico PETG con un patrón de relleno de alta densidad (40 % o superior) para absorber esfuerzos de flexión. Se integraron los actuadores en sus correspondientes articulaciones físicas: un servomotor MG996R en la base (giro izquierdo/derecha), un servomotor MG996R en el hombro (para soportar el torque gravitacional) y un servomotor MG90S en el codo.

VI. IMPLEMENTACIÓN DE SOFTWARE Y ALGORITMOS DE CONTROL

El ecosistema de software del manipulador se divide en tres capas fundamentales que operan coordinadamente de forma puramente virtual para el entorno de simulación (la interfaz web interactiva de usuario, el generador síncrono de trayectorias en ROS 2 y el puente de comunicación de red):

A. Arquitectura de Comunicación e Intercambio de Datos

El ecosistema de software opera en un esquema coordinado donde la interfaz de usuario interactiva y el resolutor dinámico de la simulación física se comunican en tiempo real a través de un puente HTTP y tópicos de ROS 2. El flujo de datos e integración de los componentes clave se describe a continuación:

- **Capa de Interfaz Gráfica (Frontend):** Implementada en HTML/JS (`app.js`), captura la interacción del operador mediante barras de desplazamiento para los ángulos articulares o entradas cartesianas tridimensionales de posición (X, Y, Z) calculadas localmente mediante cinemática inversa. Cuando ocurre un cambio, la función `sendJointsToROS()` serializa los ángulos de las articulaciones J_1, J_2, J_3 (en grados sexagesimales) dentro de un objeto JSON y realiza una petición HTTP POST throttleada (con un intervalo mínimo de 50 ms) hacia el endpoint local de la API `/api/move`.
- **Capa del Puente de Red (Web Bridge):** El script en Python `web_bridge.py` inicializa de forma concurrente un servidor HTTP en el puerto 8080 y un nodo de ROS 2. Al recibir la petición POST en la ruta `/api/move`, el manejador de peticiones del servidor decodifica el objeto JSON, recupera los valores angulares y ejecuta la función `publish_joints()` del nodo. Esta función convierte de forma instantánea las variables articulares de grados sexagesimales a radianes y las publica en el tópico `/arm_controller/commands` de ROS 2.
- **Capa de Simulación Virtual (ROS 2 / Gazebo):** El resolutor físico y dinámico de Gazebo está suscrito de forma nativa a `/arm_controller/commands`. Al recibir los radianes de consigna, los controladores de posición articulares virtuales aplican torques proporcionales para llevar los eslabones simulados del robot a la pose deseada.
- **Generación de Trayectorias Autónomas:** De manera paralela e independiente de la interfaz gráfica, el script `trajectory_generator.py` publica directamente en el tópico `/arm_controller/commands` a una frecuencia de 50 Hz. Este nodo interpola entre una secuencia programada de keyframes utilizando un perfil de velocidad cosenoidal (S-curve), permitiendo realizar validaciones dinámicas automáticas dentro del simulador físico Gazebo de forma desacoplada del frontend web.

B. Interfaz Web de Control e Interacción Dinámica (`app.js`)

La interfaz de usuario está desarrollada en JavaScript de cliente puro y HTML5. Permite realizar simulación en línea y resolutor de cinemática inversa interactiva. El algoritmo detallado de la cinemática inversa se presenta en el Código 1 en el Apéndice A. Los componentes de la capa de interfaz web realizan las siguientes tareas:

- **Lienzo de Dibujo Interactivo (Canvas):** La rutina `drawRobot()` proyecta en tiempo real el comportamiento dinámico bidimensional del robot sobre un lienzo

Canvas HTML5, mostrando vistas superior (X - Y) y lateral (X - Z) para dar retroalimentación visual al operador sobre la trayectoria de corte.

- **Interpolación de Suavizado de Movimiento:** Para transicionar sin saltos discretos hacia las coordenadas indicadas, la función `animateToJoints()` aplica un algoritmo de facilidad cúbica (*Cubic Ease-Out*) a 60 cuadros por segundo, emulando el amortiguamiento físico de los servomotores reales.

C. Generador de Trayectorias de Simulación (`trajectory_generator.py`)

El nodo generador de trayectorias en Python modela el perfil cinemático de interpolación de simulación en ROS 2 y Gazebo. El lazo periódico de control temporal de este script se detalla en el Código 2 en el Apéndice B. Este nodo realiza la generación y control síncrono mediante:

- **Interpolación Cosenoidal (S-curve):** La ecuación cosenoidal suaviza el incremento de velocidad al inicio y finalización del movimiento de cada eslabón, eliminando las aceleraciones abruptas que provocan oscilaciones inerciales o colisiones mecánicas virtuales en el resolutor de física de Gazebo.
- **Bucle Síncrono a 50 Hz:** Garantiza una comunicación continua hacia los controladores articulares virtuales a intervalos exactos de 20 ms.

D. Puente de Comunicación ROS 2 y Control de Hardware (`web_bridge.py`)

La comunicación intermedia entre la simulación virtual, la interfaz web y el prototipo físico se centraliza en un script multi-hilo desarrollado en Python. Este puente no solo traduce y publica tramas en ROS 2 para Gazebo, sino que integra el control del hardware físico real mediante las siguientes bases técnicas:

- **Servidor HTTP Multihilo y Endpoints de Hardware:** Inicializa un servidor HTTP en el puerto 8080. Para controlar el prototipo real, expone los endpoints `/api/physical/connect` (conexión UART a 115200 bps), `/api/physical/move` (envío síncrono de comandos a los servomotores), `/api/physical/list_ports` (listado y detección automática de puertos `/dev/ttyUSB*` / `/dev/ttyACM*`) y `/api/physical/status` (estado de conexión y respuesta del firmware).
- **SerialManager Thread-safe:** Implementa una clase de gestión serial con bloqueos de exclusión mutua (`threading.Lock`). Esto previene condiciones de carrera cuando se envían tramas síncronas periódicas a la planta física. El puente autodetecta el ESP32 a partir de identificadores VID/PID de conversores USB-TTL típicos (CH340, CP210x) y ofrece reconexión dinámica transparente.
- **Servicios y Streaming de Video en Tiempo Real:** Expone el endpoint `/api/list_cameras` que escanea y enumera los dispositivos de video ópticos conectados. Además, mediante `/api/real_camera_`

stream, sirve un flujo de video cenital en formato multipart/x-mixed-replace (MJPEG) con tasa constante de 16 FPS. Si la cámara real (priorizando /dev/video2) se desconecta físicamente, el endpoint dibuja dinámicamente un fotograma de error y entra en bucle de reintento automático sin colapsar el hilo servidor.

E. Procesamiento de Imagen, HSV Red Mask y Trayectoria 3D (web_bridge.py)

Para permitir que el brazo robótico físico siga trazos hechos a mano alzada en el mundo real, se diseñó un pipeline de visión artificial y reconstrucción tridimensional:

- **Detección y Homografía del Cilindro:** Utilizando OpenCV, se aísla el cuerpo del cilindro de PETG mediante un filtrado por color en el espacio HSV con una mascarilla cian (rango [75, 30, 30] a [140, 255, 255]). Se calcula su caja delimitadora rectangular para proyectar inversamente y desenrollar geométricamente el cilindro físico en un plano de visualización ($W_u \times H_u = 512 \times 256$ píxeles).
- **Segmentación del Trazo Rojo y Zhang-Suen:** Una vez desenrollada la superficie, se extrae el trazo dibujado a mano. Para ello, se aplica una mascarilla de color ****Rojo**** en el espacio HSV utilizando una unión de rangos duales para abarcar la naturaleza periódica del matiz (Hue): de [0, 50, 40] a [10, 255, 255] (límites inferiores) y de [165, 50, 40] a [180, 255, 255] (límites superiores). La máscara resultante se erosiona para remover ruido de bordes y se procesa mediante el algoritmo vectorizado de Zhang-Suen para reducir el trazo a un espesor unitario manteniendo la topología original.
- **Reconstrucción y Conversión Espacial 3D:** Los píxeles del esqueleto del trazo rojo se ordenan por vecindad y se simplifican mediante Ramer-Douglas-Peucker (RDP). Finalmente, cada píxel (p_x, p_y) se proyecta a coordenadas tridimensionales en milímetros sobre la superficie curva real mediante el mapeo inverso cilíndrico, agregando el estado de conmutación del emisor láser.

F. Orquestación y Lanzamiento de Simulación (gazebo.launch.py)

La configuración y el inicio coordinado de los nodos de simulación física en ROS 2 se gestionan mediante un archivo de lanzamiento desarrollado en Python. El flujo secuencial y la configuración del entorno se detallan en el Código 4 en el Apéndice D. Este orquestador realiza las siguientes operaciones:

- **Carga y Procesamiento Dinámico de URDF/Xacro:** El script utiliza la utilidad `xacro.process_file()` para interpretar el modelo XML parametrizado. Posteriormente, aplica expresiones regulares para remover los comentarios XML redundantes con el fin de prevenir errores en el parser de ROS 2 y evitar inconsistencias en el plugin de Gazebo.
- **Gestión de Rutas de Modelos de Gazebo:** Actualiza dinámicamente la variable de entorno

GAZEBO_MODEL_PATH para incluir el directorio local de mallas STL del brazo, garantizando la carga correcta de las geometrías de visualización en Gazebo Classic.

- **Secuenciación de Nodos mediante Manejo de Eventos:** Para evitar condiciones de carrera donde los controladores articulares intentan enlazarse antes de que el robot exista físicamente en la física del simulador, el script utiliza manejadores de eventos `RegisterEventHandler` y `OnProcessExit`. Esto fuerza que el controlador de posición de las articulaciones (`arm_controller`) se lance únicamente después de que el robot haya sido instanciado por el nodo `spawn_entity`.

G. Modelo Descriptor de la Estructura y Actuadores (arm.urdf.xacro)

El diseño estructural, los límites de movimiento y la configuración de hardware de control del gemelo virtual se definen jerárquicamente en un archivo descriptor Xacro, presentado en el Código 5 en el Apéndice E. Este descriptor se estructura a partir de los siguientes elementos clave:

- **Geometría de Eslabones y Mallas STL:** Vincula directamente los archivos STL exportados de CAD (`base.stl`, `link1.stl`, etc.) escalados a metros (factor 0,001), definiendo la apariencia visual del robot en el simulador.
- **Límites y Dinámica de Articulaciones:** Modela los límites físicos angulares en radianes equivalentes a los límites de los servomotores físicos del prototipo. Adicionalmente, incluye parámetros dinámicos de fricción y amortiguamiento (`friction` y `damping`) para modelar el rozamiento de las articulaciones en Gazebo.
- **Integración con ros2_control:** Declara el elemento `<ros2_control>` que mapea las interfaces de comando y de estado (posiciones articulares y velocidades) del resolutor de física de Gazebo Classic (`gazebo_ros2_control/GazeboSystem`), permitiendo la inyección directa de señales desde el puente de comunicación HTTP. Esta declaración se complementa en la arquitectura de ROS 2 con el archivo de configuración `controllers.yaml`, el cual especifica la tasa de actualización de los bucles de control a 100 Hz y define un controlador síncrono del tipo `JointGroupPositionController` asociado a las tres articulaciones activas del robot.

H. Automatización de Configuración y Compilación (setup.sh)

Para simplificar la inicialización del entorno y garantizar la reproducibilidad de las pruebas, se programó un script de automatización en Bash. El funcionamiento y la lógica de este script se detallan en el Código 6 en el Apéndice F. Este script realiza las siguientes operaciones secuenciales:

- **Verificación del Entorno Aislado:** Comprueba la existencia del contenedor Distrobox (`ros2-humble`) y, en

caso de no encontrarse activo, lo crea automáticamente instalando una base de Ubuntu 22.04.

- **Compilación Síncrona del Workspace:** Accede al contenedor, carga las variables de entorno de ROS 2 Humble y compila exclusivamente el paquete `arm_simulation` mediante el comando `colcon build`.
- **Generación de Scripts Lanzadores:** Escribe y otorga permisos de ejecución a los tres scripts rápidos (`launch_gazebo.sh`, `run_trajectory.sh` y `start_ui.sh`) en la raíz del proyecto para facilitar el uso diario.

I. Scripts Lanzadores del Entorno y Ejecutables

Para agilizar el flujo de trabajo del operador sin requerir la memorización de comandos de contenedor o de ROS 2, se desarrollaron tres scripts lanzadores. Su código fuente se presenta en el Código 7 en el Apéndice G. Cada script cumple una función específica dentro de la arquitectura distribuida:

- **Lanzamiento de Simulación (`launch_gazebo.sh`):** Limpia la variable de entorno `PATH` para remover rutas de Conda (previniendo conflictos de intérprete de Python con las dependencias de ROS 2), carga los entornos locales y ejecuta el launch de Gazebo.
- **Generador de Trayectorias (`run_trajectory.sh`):** Realiza la misma sanitización del entorno y ejecuta directamente el nodo controlador de interpolación cosenoidal.
- **Inicio de la Interfaz Gráfica (`start_ui.sh`):** Lanza de manera asíncrona el navegador predeterminado apuntando a `http://localhost:8080` y arranca concurrentemente el puente HTTP/ROS 2 (`web_bridge.py`) en segundo plano.

J. Firmware Embebido del Microcontrolador Físico (`main.cpp`)

Para el control directo y la coordinación de los actuadores del prototipo físico, se desarrolló un firmware embebido en C++ ejecutado sobre el microcontrolador ESP32. Su implementación completa se detalla en el Código 10 en el Apéndice H. Las principales funcionalidades de control, decodificación y seguridad implementadas se describen a continuación:

- **Configuración de Periféricos e Inicialización de Actuadores:** El firmware utiliza la biblioteca `ESP32Servo` reservando tres temporizadores de hardware a una frecuencia de control de 50 Hz para generar las señales de modulación por ancho de pulso (PWM) destinadas a los servomotores de la base (GPIO 27), hombro (GPIO 14) y codo (GPIO 21). Adicionalmente, el pin GPIO 22 se define como una salida digital para la conmutación del emisor láser.
- **Decodificación Bidireccional y Parser de Tramas Duales:** El firmware en su bucle periódico `loop()` decodifica tramas provenientes de la interfaz serial UART a una tasa de 115200 bps. El parser soporta dos modos de entrada dinámicos:

1. *Modo Cartesiano ($X:val, Y:val, Z:val$):* El microcontrolador ejecuta localmente el modelo de cinemática inversa analítico codo-arriba. Si las coordenadas se envían en milímetros (valores absolutos $> 2,0$), el firmware las escala a metros de forma automática.
2. *Modo Directo de Juntas ($J1:val, J2:val, J3:val, L:val$):* La interfaz web envía directamente los ángulos articulares en grados sexagesimales junto con el estado del láser ('L:0' o 'L:1'). El firmware los convierte a radianes y asocia el control de PWM de forma directa.

- **Conversión Articular y Mapeo PWM de Precisión:** La función `angleToMicroseconds()` toma el ángulo articular en radianes, valida y restringe su valor según los límites angulares permitidos (para evitar colisiones destructivas) y realiza el mapeo lineal al ancho de pulso:

$$t_{\mu s} = 1500 + \theta \cdot \frac{2000}{\pi} \quad (69)$$

clampando el resultado estrictamente en la banda de seguridad de $[500, 2500] \mu s$.

- **Algoritmo de Movimiento Gradual Simultáneo (`moveToTarget`):** Para evitar picos severos de corriente instantánea debido a la inercia del manipulador (que activarían el circuito de protección OCP de la fuente ATX, apagando el ESP32), se programó un perfilador de movimiento gradual. La rutina `moveToTarget()` interpola linealmente el ancho de pulso de los tres servos en pasos simultáneos máximos de $15 \mu s$ ($\approx 1,35^\circ$) espaciados por retardos temporales fijos de 15 ms, eliminando los transitorios de corriente dañinos. **Watchdog por Software y Parada de Emergencia:** Para prevenir incendios o marcas incidentales en la pieza de trabajo por fallos de comunicación en el lazo de control, se implementó un temporizador de watchdog por software. Si transcurren más de 1000 ms sin la llegada de comandos válidos, el láser se apaga automáticamente. Asimismo, ante la trama 'ESTOP' o 'EMERGENCY', la rutina `triggerEmergency()` realiza el apagado inmediato del emisor láser y desasocia ('detach()') las señales PWM para liberar la tensión en los servomotores.

K. Scripts de Validación Cinemática y Alineación de Marcos

Con el fin de verificar la corrección del URDF, el árbol de transformaciones (TF) y las ecuaciones cinemáticas antes de operar el hardware, se programaron diversos scripts de prueba en Python:

- **Prueba Cinemática y Comparación DH (`test_kinematics.py`):** Compara analíticamente dos formulaciones de Denavit-Hartenberg (una estándar y otra con desfases angulares compensados ϕ_2 y ϕ_3), validando la consistencia matemática de la cinemática directa. El código se detalla en el Código 12 en el Apéndice J.
- **Validación de Alineación TF en Gazebo (`test_tf_alignment.py`):** Se conecta como nodo de

ROS 2 para leer las transformaciones del árbol de TFs en tiempo real de la simulación activa. Comprueba que en la pose cero de calibración del robot, todos los ejes longitudinales X de las articulaciones sean coplanares y paralelos (Ángulo Pitch $\approx 0^\circ$ y desviación lateral $Y \approx 0$). Su código se presenta en el Código 13 en el Apéndice K.

- **Verificación Analítica de URDF (test_urdf_tf_alignment.py):** Realiza un análisis algebraico de las transformaciones de rotación RPY declaradas en el modelo descriptor URDF/Xacro, comprobando si los ejes longitudinales X resultan paralelos y horizontales en el plano de movimiento vertical. Su código se detalla en el Código 14 en el Apéndice L.
- **Lectura de Marcos de Referencia Activos (get_tf_distances.py):** Consulta el servicio de transformaciones de ROS 2 en tiempo real para imprimir las traslaciones y cuaterniones de rotación de cada eslabón con respecto a la base del robot. Su implementación se muestra en el Código 15 en el Apéndice M.
- **Parser de Geometría Xacro (get_gazebo_distances.py):** Analiza el árbol XML del archivo Xacro descriptor del brazo para extraer automáticamente los límites físicos, desfases de juntas y longitudes calculadas de los eslabones móviles. Su código se detalla en el Código 16 en el Apéndice N.

VII. SELECCIÓN DE ACTUADORES Y SENSORES

Para garantizar el éxito de la implementación física y su congruencia con el entorno de simulación, se detalla la selección de componentes de hardware:

A. Actuadores

La arquitectura cinemática de 3 GDL resulta idónea para este caso de estudio; dado que la herramienta es un haz láser unidireccional colineal con el eje de aproximación, y el objetivo es el corte y grabado sobre superficies cilíndricas, el control del posicionamiento cartesiano en los tres ejes (X, Y, Z) es crítico para compensar la curvatura de la pieza y mantener la distancia focal constante del haz. La omisión del control de orientación redundante (pitch/roll) disminuye drásticamente el torque estático requerido en las articulaciones proximales (base y hombro), lo que mitiga las deflexiones estructurales y maximiza la rigidez y repetibilidad del sistema. El diseño se complementa ubicando estratégicamente actuadores de baja masa en los eslabones distales para minimizar el tensor de inercia del mecanismo.

- **Actuador de rotación principal (Base):** Se implementó un servomotor de alto torque y engranajes metálicos (MG996R) en la base para accionar el giro del brazo a la izquierda y derecha. Este actuador soporta las cargas dinámicas y el momento de inercia de toda la estructura móvil.

- **Actuadores de elevación (Hombro y Codo):** Se utilizó un servomotor de alto torque MG996R para el hombro (articulación pegada a la base que hace elevar el brazo) para soportar el peso y los esfuerzos de flexión de los eslabones, y un servomotor de engranajes metálicos MG90S en la articulación del codo para controlar la elevación distal.
- **Efector final (Módulo Láser de estado sólido):** Consiste en un diodo láser de baja potencia con lógica transistor-transistor (TTL) de 5V, el cual es conmutado como un nivel lógico de entrada/salida de propósito general (GPIO) o vía PWM desde el ESP32. Esto permite una activación síncrona y de baja latencia acoplada al perfil cinemático de la trayectoria, emulando la dinámica de interpolación de un proceso industrial de corte láser CNC.

B. Sensores

- **Potenciómetros internos:** Los servomotores ya incluyen sensores de posición (potenciómetros) internamente. Esto mantiene el diseño mecánico limpio ya que no se requiere acoplar encoders ópticos externos para leer los ángulos de las articulaciones, eliminando la necesidad de cableado para sensores externos de posición o corriente.
- **Cámara física de monitoreo:** Se integra una cámara USB externa para la captura y monitoreo del proceso de guiado en tiempo real. Físicamente, el lente de la cámara se ubica fijamente a una distancia de 15 cm (150 mm) desde el origen del sistema de referencia de la base $\{0\}$ a lo largo de su eje longitudinal X_0 , y a una altura de 2,5 cm (25 mm) a lo largo del eje vertical Z_0 . Esta colocación rígida permite obtener una visualización cenital despejada de la superficie cilíndrica a mecanizar.

C. Especificación de Componentes

En la Tabla III se detallan las especificaciones y funciones de los componentes de hardware seleccionados.

Tabla III
ESPECIFICACIONES DE LOS COMPONENTES DE HARDWARE

Componente	Modelo	Función en el Sistema
Controlador	ESP32	Control de bajo nivel y PWM.
Actuador base	MG996R (1 ud.)	Rotación izquierda/derecha.
Actuador hombro	MG996R (1 ud.)	Elevación del hombro.
Actuador codo	MG90S (1 ud.)	Elevación del codo.
Efector final	Diodo Láser (5V)	Haz de corte de baja potencia.
Cámara	USB/Monitoreo	Captura visual del cilindro de trabajo.
Estructura	PETG	Piezas impresas en 3D.
Alimentación	Fuente 5V 5A	Energía para actuadores y láser.

D. Presupuesto de Masa y Dimensiones

En la Tabla IV se detalla la masa y dimensiones físicas estimadas para los diferentes componentes del brazo manipulador.

Tabla IV
PRESUPUESTO DE MASA Y DIMENSIONES DE LOS COMPONENTES

Componente	Masa (g)	Cant.	Dimensiones (mm)
ESP32 NodeMCU	10	1	48.0 × 26.0 × 12.0
Servo MG996R	55	2	40.7 × 19.7 × 42.9
Servo MG90S	13.4	1	22.8 × 12.2 × 28.5
Diodo Láser 5V	10	1	φ 12.0 × 35.0
Estructura 3D (PETG)	350	1	Envolvente ≈ 250 × 150
Fuente 5V 5A	150	1	110.0 × 78.0 × 36.0
Cables y tornillería	50	1	N/A
Masa Total	693 g	(≈ 0.69 kg)	

E. Presupuesto y Consumo de Energía

La Tabla V detalla el voltaje nominal y las corrientes de funcionamiento promedio y de pico para estimar la potencia del sistema.

Tabla V
PRESUPUESTO Y CONSUMO DE ENERGÍA DE LOS COMPONENTES

Componente	Voltaje (V)	Corr. Prom.	Corr. Pico
ESP32	5.0	120 mA	240 mA
MG996R (2 ud.)	5.0	1000 mA	5.0 A
MG90S (1 ud.)	5.0	180 mA	0.8 A
Láser 5V	5.0	30 mA	30 mA
Consumo Promedio		1.33 A (≈ 6.7 W)	
Consumo Pico		6.07 A (≈ 30.4 W)	

F. Estimación de Costos

El presupuesto estimado para la adquisición de los componentes de hardware requeridos se presenta en la Tabla VI.

Tabla VI
PRESUPUESTO Y ESTIMACIÓN DE COSTOS

Descripción	Cant.	Unit. (\$/.)	Total (\$/.)
ESP32 NodeMCU	1	20.00	20.00
Servomotor MG996R	2	26.00	52.00
Servomotor MG90S	1	8.00	8.00
Módulo Láser 5V	1	3.50	3.50
Impresión 3D (Estructura)	1	93.00	93.00
Cables y tornillería	-	10.00	10.00
Total Estimado			S/. 186.50

G. Esquema de Conexiones Eléctricas

La integración física del sistema requiere una arquitectura de alimentación y control claramente definida. El ESP32 es alimentado directamente por la laptop vía USB, mientras que los servomotores y el diodo láser reciben su tensión de operación de +5 V desde la fuente de PC. El encendido del láser se gestiona mediante un transistor NPN 2N2222 con resistencia de base de 220 Ω, accionado por el pin GPIO 25 del ESP32. La Fig. 5 muestra el esquema completo de conexiones.

VIII. APLICACIÓN INDUSTRIAL

A diferencia de los pantógrafos y cortadoras láser cartesianas convencionales, los cuales están limitados a trazar trayectorias coplanares bidimensionales (2D) en láminas planas, la configuración antropomórfica de 3 GDL permite posicionar la herramienta en un espacio tridimensional (X, Y, Z). Esta característica habilita la aplicación del robot en procesos industriales de corte y grabado sobre superficies irregulares y no coplanares, tales como el perfilado de cuerpos curvados y el calado de piezas cilíndricas.

En la industria de manufactura mecánica, el procesamiento de piezas no planas requiere realizar cortes y grabados con perfiles tridimensionales complejos (por ejemplo, intersecciones de geometrías cilíndricas o marcas en piezas curvas). Al colocar un haz láser en el efector final del robot, es posible realizar este proceso sin contacto mecánico directo. A partir del modelo de la superficie cilíndrica de una pieza de radio R_{tubo} colocada de forma horizontal a lo largo del eje Y_0 a una distancia x_{tubo} y a una altura de desfase z_{tubo} , la coordenada vertical de corte z_E se calcula dinámicamente en función de la posición radial x_E :

$$z_E = z_{tubo} + \sqrt{R_{tubo}^2 - (x_E - x_{tubo})^2} \quad (70)$$

El resolutor de cinemática inversa desarrollado en (26)-(41) calcula de manera instantánea las posiciones articulares ($\theta_1, \theta_2, \theta_3$) para que el láser siga esta trayectoria no planar, manteniendo la distancia focal constante respecto a la pared de la pieza. Esto demuestra el valor industrial del manipulador propuesto frente a las soluciones cartesianas convencionales de bajo costo, ampliando el alcance de automatización hacia geometrías complejas no convencionales.

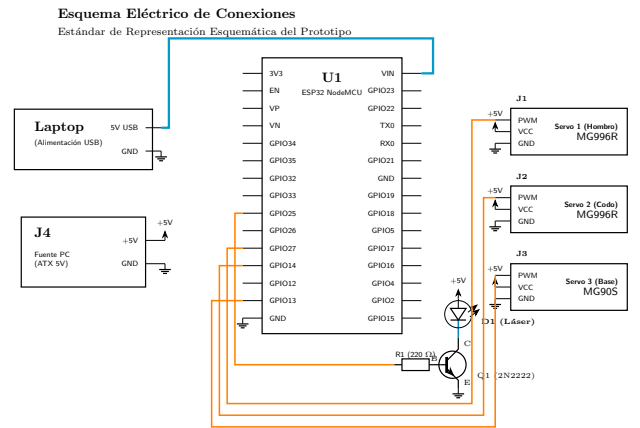


Figura 5. Esquema de conexiones eléctricas del prototipo. Se detallan las líneas de alimentación desde la fuente de PC (+5 V y GND) hacia los servomotores Servo 1 (MG996R), Servo 2 (MG996R) y Servo 3 (MG90S), el circuito de control del diodo láser con transistor NPN 2N2222 y resistencia de base de 220 Ω, y la conexión USB de alimentación del ESP32 NodeMCU de 38 pines.

APÉNDICE A

CÓDIGO DEL RESOLVEDOR DE CINEMÁTICA INVERSA EN LA INTERFAZ WEB (APP.JS)

```
1 function solveIK(x, y, z, customL2) {
2   const targetX = parseFloat(x);
3   const targetY = parseFloat(y);
4   const targetZ = parseFloat(z);
5
6   const currentL2 = (customL2 !== undefined) ? customL2 : L2;
7
8   if (isNaN(targetX) || isNaN(targetY) || isNaN(targetZ)) {
9     return { error: 'VALORES_INVALIDOS', msg: 'Ingresa valores numericos validos.' };
10  }
11
12  let r = Math.sqrt(targetX * targetX + targetY * targetY);
13  if (r < Math.abs(d4)) {
14    r = Math.abs(d4);
15  }
16  const Rp = Math.sqrt(r * r - d4 * d4);
17  const thetal = (targetX === 0 && targetY === 0) ? -Math.PI / 2 : Math.atan2(
18    targetY, targetX) - Math.atan2(-d4, Rp);
19  const j1 = thetal * 180 / Math.PI;
20
21  if (j1 < jointsConfig[1].min || j1 > jointsConfig[1].max) {
22    return { error: 'FUERA DE ALCANCE', msg: 'Rotacion base J1 fuera de limites (-165 a 165).' };
23  }
24
25  const xc = Rp - a2;
26  const zc = targetZ - (d1 + d2);
27
28  const psi_numerator = xc * xc + zc * zc - L1 * L1 - currentL2 * currentL2;
29  const psi_denominator = 2.0 * L1 * currentL2;
30  let psi = psi_numerator / psi_denominator;
31
32  if (psi > 1.0 && psi <= 1.005) {
33    psi = 1.0;
34  } else if (psi < -1.0 && psi >= -1.005) {
35    psi = -1.0;
36  }
37
38  if (psi < -1.0 || psi > 1.0) {
39    return { error: 'FUERA DE ALCANCE', msg: 'Coordenadas fuera del alcance del brazo.' };
40  }
41
42  const solutions = [-1, 1];
43  for (let s of solutions) {
44    const sinTheta3_star = s * Math.sqrt(1.0 - psi * psi);
45    const theta3_star = Math.atan2(sinTheta3_star, psi);
46
47    const theta2_star = Math.atan2(zc, xc) - Math.atan2(currentL2 * sinTheta3_star,
48      L1 + currentL2 * psi);
49
50    const theta2 = theta2_star;
51    const theta3 = theta3_star;
52
53    const j2 = theta2 * 180 / Math.PI;
54    const j3 = theta3 * 180 / Math.PI;
55
56    if (j2 >= jointsConfig[2].min && j2 <= jointsConfig[2].max &&
57      j3 >= jointsConfig[3].min && j3 <= jointsConfig[3].max) {
58      return { j1, j2, j3 };
59    }
60  }
61
62  return { error: 'FUERA DE ALCANCE', msg: 'Posicion cinematicamente inalcanzable por limites de articulacion.' };
63 }
```

Listing 1. Resolvedor interactivo de cinemática inversa en JS (app.js).

APÉNDICE B

CÓDIGO DEL GENERADOR DE TRAYECTORIAS DE SIMULACIÓN (TRAJECTORY_GENERATOR.PY)

```
1 #!/usr/bin/env python3
2 import rclpy
3 from rclpy.node import Node
4 from std_msgs.msg import Float64MultiArray
5 import time
6 import math
7
8 class TrajectoryGenerator(Node):
9     def __init__(self):
10         """
11         Inicializa el nodo de ROS 2, configura el publicador de comandos de control
12         , define los keyframes de la secuencia de corte industrial y arranca el
13         temporizador de interpolacion.
14         """
15         super().__init__('trajectory_generator')
16         self.publisher_ = self.create_publisher(Float64MultiArray, '/arm_controller
17 /commands', 10)
18
19         self.rate = 50.0
20         self.timer = self.create_timer(1.0 / self.rate, self.timer_callback)
21
22         self.keyframes = [
23             {"pose": [0.0, 0.0, 0.0], "time": 3.0},
24             {"pose": [-0.6, 0.4, -0.4], "time": 3.0},
25             {"pose": [0.6, 0.4, -0.4], "time": 4.0},
```

```
26             {"pose": [0.6, 0.6, -0.7], "time": 2.0},
27             {"pose": [-0.6, 0.6, -0.7], "time": 4.0},
28             {"pose": [0.0, 0.0, 0.0], "time": 3.0}
29 ]
30
31 self.current_keyframe_idx = 0
32 self.step_counter = 0
33 self.total_steps_for_transition = 0
34 self.start_pose = [0.0, 0.0, 0.0]
35 self.target_pose = self.keyframes[0]["pose"]
36
37 self.get_logger().info("Nodo de trayectoria inicializado. Iniciando ciclo
38 de corte...")
39 self.prepare_next_transition()
40
41 def prepare_next_transition(self):
42     """
43     Prepara los parametros cinematicos y de interpolacion para la transicion
44     hacia el siguiente keyframe de la secuencia, incrementando el indice de trayectoria.
45     """
46     self.start_pose = list(self.target_pose)
47     keyframe = self.keyframes[self.current_keyframe_idx]
48     self.target_pose = keyframe["pose"]
49     duration = keyframe["time"]
50
51     self.total_steps_for_transition = int(duration * self.rate)
52     self.step_counter = 0
53
54     self.get_logger().info(
55         f"Transicionando a Keyframe {self.current_keyframe_idx}: {self.
56         target_pose} en {duration}s"
57     )
58
59     self.current_keyframe_idx = (self.current_keyframe_idx + 1) % len(self.
60     keyframes)
61
62 def timer_callback(self):
63     """
64     Ejecuta periodicamente la interpolacion cosenoidal (S-curve) entre poses
65     conjuntas sucesivas y publica el vector de comandos correspondiente a la tasa del
66     bucle.
67     """
68     if self.step_counter >= self.total_steps_for_transition:
69         self.prepare_next_transition()
70
71     t = float(self.step_counter) / float(self.total_steps_for_transition)
72     smooth_t = (1.0 - math.cos(t * math.pi)) / 2.0
73
74     current_command = []
75     for start, target in zip(self.start_pose, self.target_pose):
76         val = start + (target - start) * smooth_t
77         current_command.append(val)
78
79     msg = Float64MultiArray()
80     msg.data = current_command
81     self.publisher_.publish(msg)
82
83     self.step_counter += 1
84
85 def main(args=None):
86     """
87     Funcion de entrada que inicializa el entorno de ROS 2, instancia el generador
88     de trayectoria, mantiene el ciclo de ejecucion activo y maneja el apagado limpio del nodo.
89     """
90     rclpy.init(args=args)
91     node = TrajectoryGenerator()
92     try:
93         rclpy.spin(node)
94     except KeyboardInterrupt:
95         node.get_logger().info("Generador de trayectoria detenido por el usuario.")
96     finally:
97         node.destroy_node()
98         rclpy.shutdown()
99
100 if __name__ == '__main__':
101     main()
```

Listing 2. Generación de perfiles S-curve por interpolación cosenoidal y control de keyframes (trajectory_generator.py).

APÉNDICE C

CÓDIGO DEL PUENTE DE COMUNICACIÓN ROS 2 - INTERFAZ WEB (WEB_BRIDGE.PY)

```
1 def publish_joints(self, j1_deg, j2_deg, j3_deg):
2     j1_rad = float(j1_deg) * math.pi / 180.0
3     j2_rad = float(j2_deg) * math.pi / 180.0
4     j3_rad = float(j3_deg) * math.pi / 180.0
5     msg = Float64MultiArray()
6     msg.data = [j1_rad, j2_rad, j3_rad]
7     self.publisher_.publish(msg)
8     self.get_logger().info(f'Publicado en ROS -> J1: {j1_rad:.4f} rad, J2: {j2_rad
9     :.4f} rad, J3: {j3_rad:.4f} rad')
```

Listing 3. Conversión de unidades y envío al simulador (web_bridge.py).

APÉNDICE D

CÓDIGO DEL LANZAMIENTO DE SIMULACIÓN EN ROS 2 (GAZEBO.LAUNCH.PY)

```
1 import os
2 from ament_index_python.packages import get_package_share_directory
3 from launch import LaunchDescription
4 from launch.actions import IncludeLaunchDescription, RegisterEventHandler
5 from launch.event_handlers import OnProcessExit
6 from launch.launch_description_sources import PythonLaunchDescriptionSource
7 from launch_ros.actions import Node
8 import xacro
9
10 def generate_launch_description():
11     pkg_share = get_package_share_directory('arm_simulation')
12     xacro_file = os.path.join(pkg_share, 'urdf', 'arm.urdf.xacro')
13
14     pkg_share_parent = os.path.dirname(pkg_share)
15     default_gazebo_models = '/usr/share/gazebo-11/models:/usr/share/gazebo/models'
16     if 'GAZEBO_MODEL_PATH' in os.environ:
17         os.environ['GAZEBO_MODEL_PATH'] = pkg_share_parent + ':' +
18         default_gazebo_models + ':' + os.environ['GAZEBO_MODEL_PATH']
19     else:
20         os.environ['GAZEBO_MODEL_PATH'] = pkg_share_parent + ':' +
21         default_gazebo_models
22
23     os.environ['GAZEBO_MODEL_DATABASE_URI'] = ''
24
25     import re
26     robot_description_config = xacro.process_file(xacro_file)
27     robot_desc = robot_description_config.toxml()
28     robot_desc = re.sub(r'<!--.*?-->', '', robot_desc, flags=re.DOTALL)
29
30     robot_state_publisher_node = Node(
31         package='robot_state_publisher',
32         executable='robot_state_publisher',
33         name='robot_state_publisher',
34         output='screen',
35         parameters=[{'robot_description': robot_desc, 'use_sim_time': True}]
36     )
37
38     gazebo = IncludeLaunchDescription(
39         PythonLaunchDescriptionSource([os.path.join(
40             get_package_share_directory('gazebo_ros'), 'launch', 'gazebo.launch.py'
41         )]),
42         launch_arguments={'verbose': 'true'}.items()
43     )
44
45     spawn_entity = Node(
46         package='gazebo_ros',
47         executable='spawn_entity.py',
48         arguments=['-topic', 'robot_description', '-entity', 'arm_3gdl', '-timeout',
49             '120'],
50         output='screen'
51     )
52
53     spawn_joint_state_broadcaster = Node(
54         package='controller_manager',
55         executable='spawner',
56         arguments=['joint_state_broadcaster', '--controller-manager', '/
57             controller_manager'],
58         output='screen'
59     )
60
61     spawn_arm_controller = Node(
62         package='controller_manager',
63         executable='spawner',
64         arguments=['arm_controller', '--controller-manager', '/controller_manager'
65             ],
66         output='screen'
67     )
68
69     return LaunchDescription([
70         gazebo,
71         robot_state_publisher_node,
72         spawn_entity,
73         RegisterEventHandler(
74             event_handler=OnProcessExit(
75                 target_action=spawn_entity,
76                 on_exit=[spawn_joint_state_broadcaster],
77             )
78         ),
79         RegisterEventHandler(
80             event_handler=OnProcessExit(
81                 target_action=spawn_joint_state_broadcaster,
82                 on_exit=[spawn_arm_controller],
83             )
84         )
85     ])
```

Listing 4. Script de lanzamiento y control secuencial en ROS 2
(gazebo.launch.py).

APÉNDICE E

ARCHIVO DE DESCRIPCIÓN ESTRUCTURAL Y ACTUADORES XACRO (ARM.URDF.XACRO)

```
1 <?xml version="1.0"?>
2 <robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="arm_3gdl">
3
4 <!-- ===== Materiales de Visualizacion ===== -->
5 <material name="abb_orange">
6   <color rgba="1.0 0.45 0.0 1.0"/>
7 </material>
```

```
8 <material name="abb_black">
9   <color rgba="0.12 0.12 0.12 1.0"/>
10 </material>
11 <material name="silver">
12   <color rgba="0.75 0.75 0.75 1.0"/>
13 </material>
14 <material name="graphite">
15   <color rgba="0.25 0.25 0.25 1.0"/>
16 </material>
17 <material name="brass">
18   <color rgba="0.8 0.6 0.2 1.0"/>
19 </material>
20 <material name="laser_red">
21   <color rgba="1.0 0.0 0.0 0.8"/>
22 </material>
23
24 <!-- ===== Base Fija (Mundo) ===== -->
25 <link name="world"/>
26 <joint name="world_to_base" type="fixed">
27   <parent link="world"/>
28   <child link="base_link"/>
29   <origin xyz="0 0 0" rpy="1.57079632679 0 0"/>
30 </joint>
31
32 <!-- ===== Eslabon Base ===== -->
33 <link name="base_link">
34   <inertial>
35     <mass value="1.0"/>
36     <origin xyz="0 0 0" rpy="0 0 0"/>
37     <inertia ixx="0.005" ixy="0.0" ixz="0.0" iyy="0.005" iyz="0.0" izz="0.005"/>
38   </inertial>
39   <visual>
40     <origin xyz="-0.691316 -0.869483 -1.350860" rpy="0 0 0"/>
41     <geometry>
42       <mesh filename="package://arm_simulation/meshes/custom_arm/base.stl" scale=
43         "0.001 0.001 0.001"/>
44     </geometry>
45     <material name="abb_orange"/>
46   </visual>
47 </link>
48
49 <!-- ===== Articulacion 1 (Rotacion de Base) ===== -->
50 <joint name="joint1" type="revolute">
51   <origin rpy="-1.57079632679 0 0" xyz="0 0.060 0"/>
52   <parent link="base_link"/>
53   <child link="link1"/>
54   <axis xyz="0 0 1"/>
55   <limit lower="-2.87979" upper="2.87979" effort="20.0" velocity="1.0"/>
56   <dynamics damping="1.0" friction="1.0"/>
57 </joint>
58
59 <!-- ===== Eslabon 1 ===== -->
60 <link name="link1">
61   <inertial>
62     <mass value="0.5"/>
63     <origin xyz="0 0 0" rpy="0 0 0"/>
64     <inertia ixx="0.002" ixy="0.0" ixz="0.0" iyy="0.002" iyz="0.0" izz="0.002"/>
65   </inertial>
66   <visual>
67     <origin xyz="-0.691316 1.350860 -0.929483" rpy="1.57079632679 0 0"/>
68     <geometry>
69       <mesh filename="package://arm_simulation/meshes/custom_arm/link1.stl" scale
70         ="0.001 0.001 0.001"/>
71     </geometry>
72     <material name="abb_orange"/>
73   </visual>
74 </link>
75
76 <!-- ===== Articulacion 2 (Hombro) ===== -->
77 <joint name="joint2" type="revolute">
78   <origin rpy="1.57079632679 0 0" xyz="0.015 0 0.036"/>
79   <parent link="link1"/>
80   <child link="link2"/>
81   <axis xyz="0 0 1"/>
82   <limit lower="-1.91986" upper="1.91986" effort="25.0" velocity="1.0"/>
83   <dynamics damping="1.0" friction="1.0"/>
84 </joint>
85
86 <!-- ===== Eslabon 2 ===== -->
87 <link name="link2">
88   <inertial>
89     <mass value="0.5"/>
90     <origin xyz="0 0 0" rpy="0 0 0"/>
91     <inertia ixx="0.002" ixy="0.0" ixz="0.0" iyy="0.002" iyz="0.0" izz="0.002"/>
92   </inertial>
93   <visual>
94     <origin xyz="-0.706231 -0.965655 -1.350860" rpy="0 0 0"/>
95     <geometry>
96       <mesh filename="package://arm_simulation/meshes/custom_arm/link2.stl" scale
97         ="0.001 0.001 0.001"/>
98     </geometry>
99     <material name="abb_orange"/>
100   </visual>
101 </link>
102
103 <!-- ===== Articulacion 3 (Codo) ===== -->
104 <joint name="joint3" type="revolute">
105   <origin rpy="0 0 0" xyz="0.075158 0.125169 0"/>
106   <parent link="link2"/>
107   <child link="link3"/>
108   <axis xyz="0 0 1"/>
109   <limit lower="-1.91986" upper="1.22173" effort="20.0" velocity="1.5"/>
110   <dynamics damping="1.0" friction="1.0"/>
111 </joint>
112
113 <!-- ===== Eslabon 3 ===== -->
114 <link name="link3">
115   <inertial>
116     <mass value="0.3"/>
117     <origin xyz="0 0 0" rpy="0 0 0"/>
```

```

115 <inertia ixx="0.001" ixy="0.0" ixz="0.0" iyy="0.001" iyz="0.0" izz="0.001"/> 221
116 </inertial> 222
117 <visual> 223
118 <origin xyz="-0.781487 -1.090987 -1.350860" rpy="0 0 0"/> 224
119 <geometry> 225
120 <mesh filename="package://arm_simulation/meshes/custom_arm/link3.stl" scale 226
    =0.001 0.001 0.001"/>
121 </geometry> 227
122 <material name="abb_orange"/> 228
123 </visual> 229
124 </link> 230
125 231
126 <!-- ===== Acoplamiento de Herramienta (Flange y Tool0) 232
    ===== -->
127 <link name="flange"/> 233
128 <joint type="fixed" name="joint_3-flange"> 234
129 <origin xyz="0.158174 -0.030033 -0.007" rpy="0 0 -0.191392"/> 235
130 <parent link="link3"/> 236
131 <child link="flange"/> 237
132 </joint> 238
133 239
134 <link name="tool0"/> 240
135 <joint name="flange-tool0" type="fixed"> 241
136 <origin xyz="0 0 0" rpy="0 1.57079632679 0" /> 242
137 <parent link="flange" /> 243
138 <child link="tool0" /> 244
139 </joint> 245
140 246
141 <!-- ===== Controladores y Plugins de Gazebo ===== 247
    --> 248
142 <gazebo> 249
143 <plugin name="gazebo_ros2_control" filename="libgazebo_ros2_control.so"> 250
144 <parameters>$(find arm_simulation)/config/controllers.yaml</parameters> 251
145 </plugin> 252
146 </gazebo> 253
147 254
148 <ros2_control name="GazeboSystem" type="system"> 255
149 <hardware> 256
150 <plugin>gazebo_ros2_control/GazeboSystem</plugin> 257
151 </hardware> 258
152 <joint name="joint1"> 259
153 <command_interface name="position"> 260
154 <param name="min">-2.87979</param> 261
155 <param name="max">2.87979</param> 262
156 </command_interface> 263
157 <state_interface name="position"> 264
158 <param name="initial_value">0.0</param> 265
159 </state_interface>
160 <state_interface name="velocity"/>
161 </joint>
162 <joint name="joint2">
163 <command_interface name="position">
164 <param name="min">-1.91986</param>
165 <param name="max">1.91986</param>
166 </command_interface>
167 <state_interface name="position">
168 <param name="initial_value">0.0</param>
169 </state_interface>
170 <state_interface name="velocity"/>
171 </joint>
172 <joint name="joint3">
173 <command_interface name="position">
174 <param name="min">-1.91986</param>
175 <param name="max">1.22173</param>
176 </command_interface>
177 <state_interface name="position">
178 <param name="initial_value">0.0</param>
179 </state_interface>
180 <state_interface name="velocity"/>
181 </joint>
182 </ros2_control>
183 266
184 <!-- ===== Macro para Sistemas de Referencia (RGB) 267
    ===== -->
185 <xacro:macro name="visualize_frame" params="parent prefix xyz:=0 0 0 rpy:=0 0 0 268
    0 scale:=0.05">
186 <!-- Eje X (Rojo) --> 269
187 <link name="$(prefix)_frame_x"> 270
188 <inertial> 271
189 <mass value="0.0001"/> 272
190 <origin xyz="0 0 0" rpy="0 0 0"/> 273
191 <inertia ixx="1e-9" ixy="0" ixz="0" iyy="1e-9" iyz="0" izz="1e-9"/> 274
192 </inertial> 275
193 <visual> 276
194 <origin xyz="$(scale/4) 0 0" rpy="0 1.57079632679 0"/> 277
195 <geometry> 278
196 <cylinder radius="$(scale*0.01)" length="$(scale/2)"/> 279
197 </geometry> 280
198 <material name="red"> 281
199 <color rgba="1.0 0.0 0.0 0.8"/> 282
200 </material> 283
201 </visual> 284
202 </link> 285
203 <joint name="$(prefix)_frame_x_joint" type="fixed"> 286
204 <origin xyz="$(xyz)" rpy="$(rpy)"/> 287
205 <parent link="$(parent)"/> 288
206 <child link="$(prefix)_frame_x"/> 289
207 </joint> 290
208 <gazebo reference="$(prefix)_frame_x"> 291
209 <material>Gazebo/Red</material> 292
210 </gazebo> 293
211 294
212 <!-- Eje Y (Verde) --> 295
213 <link name="$(prefix)_frame_y"> 296
214 <inertial> 297
215 <mass value="0.0001"/> 298
216 <origin xyz="0 0 0" rpy="0 0 0"/> 299
217 <inertia ixx="1e-9" ixy="0" ixz="0" iyy="1e-9" iyz="0" izz="1e-9"/> 300
218 </inertial> 301
219 <visual> 302
220 <origin xyz="0 $(scale/4) 0" rpy="-1.57079632679 0 0"/> 303
221 304
222 <geometry> 305
223 <cylinder radius="$(scale*0.01)" length="$(scale/2)"/> 306
224 </geometry> 307
225 <material name="green"> 308
226 <color rgba="0.0 1.0 0.0 0.8"/> 309
227 </material> 310
228 </visual> 311
229 </link> 312
230 <joint name="$(prefix)_frame_y_joint" type="fixed"> 313
231 <origin xyz="$(xyz)" rpy="$(rpy)"/> 314
232 <parent link="$(parent)"/> 315
233 <child link="$(prefix)_frame_y"/> 316
234 </joint> 317
235 <gazebo reference="$(prefix)_frame_y"> 318
236 <material>Gazebo/Green</material> 319
237 </gazebo> 320
238 321
239 <!-- Eje Z (Azul) --> 322
240 <link name="$(prefix)_frame_z"> 323
241 <inertial> 324
242 <mass value="0.0001"/> 325
243 <origin xyz="0 0 0" rpy="0 0 0"/> 326
244 <inertia ixx="1e-9" ixy="0" ixz="0" iyy="1e-9" iyz="0" izz="1e-9"/> 327
245 </inertial> 328
246 <visual> 329
247 <origin xyz="0 0 $(scale/4)" rpy="0 0 0"/> 330
248 <geometry> 331
249 <cylinder radius="$(scale*0.01)" length="$(scale/2)"/> 332
250 </geometry> 333
251 <material name="blue"> 334
252 <color rgba="0.0 0.0 1.0 0.8"/> 335
253 </material> 336
254 </visual> 337
255 </link> 338
256 <joint name="$(prefix)_frame_z_joint" type="fixed"> 339
257 <origin xyz="$(xyz)" rpy="$(rpy)"/> 340
258 <parent link="$(parent)"/> 341
259 <child link="$(prefix)_frame_z"/> 342
260 </joint> 343
261 <gazebo reference="$(prefix)_frame_z"> 344
262 <material>Gazebo/Blue</material> 345
263 </gazebo> 346
264 </xacro:macro> 347
265 348
266 <!-- Instanciacion de los Sistemas de Referencia en cada Articulacion (D-H 349
    Secuencial) -->
267 <xacro:visualize_frame parent="base_link" prefix="system0" xyz="0 0 0" rpy=" 350
    -1.57079632679 0 0" scale="0.08"/>
268 <xacro:visualize_frame parent="link1" prefix="system1" xyz="0 0 0" rpy="0 0 0 351
    scale="0.06"/>
269 <xacro:visualize_frame parent="link2" prefix="system2" xyz="0 0 0" rpy="0 0 352
    1.030041" scale="0.06"/>
270 <xacro:visualize_frame parent="link3" prefix="system3" xyz="0 0 0" rpy="0 0 353
    -0.191392" scale="0.06"/>
271 <xacro:visualize_frame parent="flange" prefix="system4" xyz="0 0 0" rpy="0 0 0 354
    scale="0.06"/>
272 355
273 <!-- ===== Camara de Monitoreo (Gazebo) ===== --> 356
274 <link name="camera_link"> 357
275 <inertial> 358
276 <mass value="0.1"/> 359
277 <origin xyz="0 0 0" rpy="0 0 0"/> 360
278 <inertia ixx="1e-4" ixy="0" ixz="0" iyy="1e-4" iyz="0" izz="1e-4"/> 361
279 </inertial> 362
280 <visual> 363
281 <!-- Cuerpo de la camara --> 364
282 <origin xyz="0 0 0" rpy="0 0 0"/> 365
283 <geometry> 366
284 <box size="0.05 0.05 0.05"/> 367
285 </geometry> 368
286 <material name="graphite"/> 369
287 </visual> 370
288 </link> 371
289 <!-- Lente de la camara --> 372
290 <origin xyz="0.03 0 0" rpy="0 1.57079632679 0"/> 373
291 <geometry> 374
292 <cylinder radius="0.015" length="0.02"/> 375
293 </geometry> 376
294 <material name="abb_black"/> 377
295 </visual> 378
296 </link> 379
297 <joint name="camera_joint" type="fixed"> 380
298 <!-- Ubicada sobre el eje X positivo de system0, conectada a base_link para 381
    evitar fallos de spawn en Gazebo -->
299 <origin xyz="0.11 0.025 0.0" rpy="-1.57079632679 0 0"/> 382
300 <parent link="base_link"/> 383
301 <child link="camera_link"/> 384
302 </joint> 385
303 386
304 <!-- Plugin de Gazebo para transmitir la imagen en ROS 2 --> 387
305 <gazebo reference="camera_link"> 388
306 <sensor type="camera" name="work_camera"> 389
307 <update_rate>15.0</update_rate> 390
308 <visualize>true</visualize> 391
309 <camera name="head"> 392
310 <horizontal_fov>1.085595</horizontal_fov> 393
311 <image> 394
312 <width>640</width> 395
313 <height>480</height> 396
314 <format>R8G8B8</format> 397
315 </image> 398
316 <clip> 399
317 <near>0.05</near> 400
318 <far>8.0</far> 401
319 </clip> 402
320 </camera> 403
321 <plugin name="camera_controller" filename="libgazebo_ros_camera.so"> 404
322 <frame_name>camera_link</frame_name> 405
323 </plugin> 406
324 </sensor> 407

```

```

325 </gazebo>
326
327 <!-- Sistemas de referencia de la camara (Base y Lente) -->
328 <xacro:visualize_frame parent="base_link" prefix="camera_base" xyz="0.11 0.0 0.0"
    rpy="-1.57079632679 0 0" scale="0.04"/>
329 <xacro:visualize_frame parent="base_link" prefix="camera_lens" xyz="0.15 0.025
    0.0" rpy="-1.57079632679 0 0" scale="0.02"/>
330
331 </robot>

```

Listing 5. Modelado estructural, cinemático y configuración de ros2_control en Xacro (arm.urdf.xacro).

```

9 fi
10
11 distrobox enter ros2-humble -- bash -c "
12 export PATH=$(echo $PATH | tr ':' '\n' | grep -v miniconda | tr '\n' ':')
13 source /opt/ros/humble/setup.bash
14 source '$SCRIPT_DIR/ros2_ws/install/setup.bash'
15 python3 '$SCRIPT_DIR/ui/web_bridge.py'
16 "

```

Listing 9. Script de inicio del servidor y de la interfaz de control (start_ui.sh).

APÉNDICE F

SCRIPT DE AUTOMATIZACION DE CONFIGURACION Y COMPILACION (SETUP.SH)

```

1 #!/bin/bash
2 set -e
3 GREEN='\033[0;32m'
4 BLUE='\033[0;34m'
5 YELLOW='\033[1;33m'
6 RED='\033[0;31m'
7 NC='\033[0m'
8
9 echo -e "${BLUE}===== ${NC}"
10 echo -e "${GREEN}          KiraOne - SETUP & COMPILATION SYSTEM          ${NC}"
11 echo -e "${BLUE}===== ${NC}"
12
13 if ! command -v distrobox &> /dev/null; then
14     echo -e "${RED}Error: distrobox no esta instalado. ${NC}"
15     exit 1
16 fi
17
18 echo -e "${BLUE}[1/4] Verificando contenedor Distrobox... ${NC}"
19 if ! distrobox list | grep -q "ros2-humble"; then
20     distrobox create -n ros2-humble -i ubuntu:22.04 -y
21 fi
22 echo -e "${GREEN}[OK] Contenedor 'ros2-humble' detectado. ${NC}"
23
24 echo -e "${BLUE}[2/4] Compilando Workspace de ROS2... ${NC}"
25 SCRIPT_DIR="$( cd "$( dirname "${BASH_SOURCE[0]}" )" &> /dev/null && pwd )"
26
27 distrobox enter ros2-humble -- bash -c "
28     cd '$SCRIPT_DIR/ros2_ws' && \
29     source /opt/ros/humble/setup.bash && \
30     colcon build --packages-select arm_simulation
31 "
32 echo -e "${GREEN}[OK] Workspace compilado exitosamente. ${NC}"
33
34 echo -e "${BLUE}[3/4] Configurando permisos de ejecucion... ${NC}"
35 chmod +x "$SCRIPT_DIR/ros2_ws/src/arm_simulation/scripts/trajectory_generator.py"
36 echo -e "${GREEN}[OK] Permisos configurados. ${NC}"
37
38 echo -e "${BLUE}[4/4] Generando scripts de ejecucion rapida... ${NC}"
39 # Generacion automatica de launch_gazebo.sh, run_trajectory.sh y start_ui.sh ...

```

Listing 6. Script de inicialización y compilación del entorno (setup.sh).

APÉNDICE G

LANZADORES RÁPIDOS DE LA SIMULACIÓN Y DE LA INTERFAZ WEB

```

1 #!/bin/bash
2 SCRIPT_DIR="$( cd "$( dirname "${BASH_SOURCE[0]}" )" &> /dev/null && pwd )"
3 echo "Iniciando Gazebo Classic (KiraOne)..."
4 distrobox enter ros2-humble -- bash -c "
5     export PATH=$(echo $PATH | tr ':' '\n' | grep -v miniconda | tr '\n' ':')
6     source /opt/ros/humble/setup.bash
7     source '$SCRIPT_DIR/ros2_ws/install/setup.bash'
8     ros2 launch arm_simulation gazebo.launch.py
9 "

```

Listing 7. Script lanzador de Gazebo Classic (launch_gazebo.sh).

```

1 #!/bin/bash
2 SCRIPT_DIR="$( cd "$( dirname "${BASH_SOURCE[0]}" )" &> /dev/null && pwd )"
3 echo "Ejecutando generador de trayectoria (KiraOne)..."
4 distrobox enter ros2-humble -- bash -c "
5     export PATH=$(echo $PATH | tr ':' '\n' | grep -v miniconda | tr '\n' ':')
6     source /opt/ros/humble/setup.bash
7     source '$SCRIPT_DIR/ros2_ws/install/setup.bash'
8     /usr/bin/python3 '$SCRIPT_DIR/ros2_ws/src/arm_simulation/scripts/
    trajectory_generator.py'
9 "

```

Listing 8. Script ejecutor de la trayectoria cosenoidal (run_trajectory.sh).

```

1 #!/bin/bash
2 SCRIPT_DIR="$( cd "$( dirname "${BASH_SOURCE[0]}" )" &> /dev/null && pwd )"
3 echo "Iniciando servidor web y puente ROS 2 para KiraOne en http://localhost
    :8080..."
4
5 if command -v xdg-open &> /dev/null; then
6     (sleep 1.5 && xdg-open "http://localhost:8080") &
7 elif command -v sensible-browser &> /dev/null; then
8     (sleep 1.5 && sensible-browser "http://localhost:8080") &

```

APÉNDICE H

CÓDIGO DEL FIRMWARE DE CONTROL DE BAJO NIVEL Y LÁSER EN EL ESP32 (MAIN.CPP)

```

1 #include <Arduino.h>
2 #include <ESP32Servo.h>
3 #include <math.h>
4
5 #define SERVO_BASE_PIN      27
6 #define SERVO_SHOULDER_PIN  14
7 #define SERVO_ELBOW_PIN     21
8 #define LASER_PIN           22
9
10 // Constantes fisicas del brazo (metros)
11 const float L0 = 0.096f;
12 const float L1 = 0.146f;
13 const float L2 = 0.161f;
14
15 // Limites de las articulaciones (radianes)
16 const float MIN_THETA1 = -1.5708f; // -90 grados
17 const float MAX_THETA1 = 1.5708f; // +90 grados
18 const float MIN_THETA2 = -0.1745f; // -10 grados
19 const float MAX_THETA2 = 1.5708f; // +90 grados
20 const float MIN_THETA3 = -2.0944f; // -120 grados
21 const float MAX_THETA3 = 0.1745f; // +10 grados
22
23 const unsigned long SERIAL_TIMEOUT_MS = 1000;
24
25 Servo servoBase;
26 Servo servoShoulder;
27 Servo servoElbow;
28
29 // Variables de estado (microsegundos correspondientes a la posicion del servo)
30 int currentBaseUs = 1500;
31 int currentShoulderUs = 1500;
32 int currentElbowUs = 1500;
33
34 unsigned long lastPacketTime = 0;
35 bool isEmergency = false;
36
37 /*
38  * Convierte un angulo articular en radianes al correspondiente ancho de pulso PWM
39  * en microsegundos,
40  * limitando el resultado entre los margenes fisicos y de seguridad del servomotor.
41  */
42 int angleToMicroseconds(float angle_rad, float min_rad, float max_rad) {
43     float clamped = constrain(angle_rad, min_rad, max_rad);
44     float us = 1500.0f + clamped * (2000.0f / PI);
45     return constrain((int)us, 500, 2500);
46 }
47
48 /*
49  * Resuelve analiticamente el modelo cinematico inverso para un brazo de 3 GDL bajo
50  * la configuracion geometrica de codo arriba a partir de una posicion cartesiana (
51  * x, y, z).
52  */
53 bool solveInverseKinematics(float x, float y, float z, float &t1, float &t2, float
    &t3) {
54     t1 = atan2(y, x);
55
56     float r = sqrt(x * x + y * y);
57     float z_prime = z - L0;
58
59     float psi_numerator = x * x + y * y + z_prime * z_prime - L1 * L1 - L2 * L2;
60     float psi_denominator = 2.0f * L1 * L2;
61     float psi = psi_numerator / psi_denominator;
62
63     if (psi < -1.0f || psi > 1.0f) {
64         return false;
65     }
66
67     t3 = atan2(-sqrt(1.0f - psi * psi), psi);
68
69     t2 = atan2(z_prime, r) - atan2(L2 * sin(t3), L1 + L2 * cos(t3));
70
71     return true;
72 }
73
74 /*
75  * Activa de forma inmediata el estado de emergencia, apaga el emisor laser de
76  * corte
77  * y desacopla la senal de los servomotores para liberar la tension de la
78  * estructura.
79  */
80 void triggerEmergency(const char* reason) {
81     isEmergency = true;
82     digitalWrite(LASER_PIN, LOW);
83     servoBase.detach();
84     servoShoulder.detach();
85     servoElbow.detach();
86
87     Serial.print("\n=== PARADA DE EMERGENCIA ACTIVA ===\nMotivo: ");
88     Serial.println(reason);

```

```

86 Serial.println("Reinicie el ESP32 para reestablecer el sistema.");
87 }
88
89 /*
90 * Mueve todos los servos gradualmente en pasos simultaneos para evitar picos de
91 corriente
92 * bruscos que provoquen el apagado o proteccion de la fuente de alimentacion ATX.
93 */
94 void moveToTarget(int targetBaseUs, int targetShoulderUs, int targetElbowUs, int
95 stepUs = 15, int stepDelay = 15) {
96     bool baseMoving = true;
97     bool shoulderMoving = true;
98     bool elbowMoving = true;
99     while (baseMoving || shoulderMoving || elbowMoving) {
100         if (isEmergency) {
101             return;
102         }
103         // Alimentar el temporizador del watchdog de seguridad durante el recorrido
104         lastPacketTime = millis();
105
106         baseMoving = (currentBaseUs != targetBaseUs);
107         shoulderMoving = (currentShoulderUs != targetShoulderUs);
108         elbowMoving = (currentElbowUs != targetElbowUs);
109
110         if (baseMoving) {
111             if (abs(targetBaseUs - currentBaseUs) <= stepUs) {
112                 currentBaseUs = targetBaseUs;
113             } else {
114                 currentBaseUs += (targetBaseUs > currentBaseUs) ? stepUs : -stepUs;
115             }
116             servoBase.writeMicroseconds(currentBaseUs);
117         }
118
119         if (shoulderMoving) {
120             if (abs(targetShoulderUs - currentShoulderUs) <= stepUs) {
121                 currentShoulderUs = targetShoulderUs;
122             } else {
123                 currentShoulderUs += (targetShoulderUs > currentShoulderUs) ? stepUs : -
124 stepUs;
125             }
126             servoShoulder.writeMicroseconds(currentShoulderUs);
127         }
128
129         if (elbowMoving) {
130             if (abs(targetElbowUs - currentElbowUs) <= stepUs) {
131                 currentElbowUs = targetElbowUs;
132             } else {
133                 currentElbowUs += (targetElbowUs > currentElbowUs) ? stepUs : -stepUs;
134             }
135             servoElbow.writeMicroseconds(currentElbowUs);
136         }
137
138         if (baseMoving || shoulderMoving || elbowMoving) {
139             delay(stepDelay);
140         }
141     }
142
143     void setup() {
144         Serial.begin(115200);
145         delay(500);
146         Serial.println("\n=== ESP32 3-GDL Laser Arm Controller (Gradual Move) ===");
147
148         pinMode(LASER_PIN, OUTPUT);
149         digitalWrite(LASER_PIN, LOW);
150
151         ESP32PWM::allocateTimer(0);
152         ESP32PWM::allocateTimer(1);
153         ESP32PWM::allocateTimer(2);
154
155         servoBase.setPeriodHertz(50);
156         servoShoulder.setPeriodHertz(50);
157         servoElbow.setPeriodHertz(50);
158
159         // Inicializar servos y asociar pines
160         servoBase.attach(SERVO_BASE_PIN, 500, 2500);
161         servoShoulder.attach(SERVO_SHOULDER_PIN, 500, 2500);
162         servoElbow.attach(SERVO_ELBOW_PIN, 500, 2500);
163
164         // Establecer posicion central por defecto al encender
165         servoBase.writeMicroseconds(currentBaseUs);
166         servoShoulder.writeMicroseconds(currentShoulderUs);
167         servoElbow.writeMicroseconds(currentElbowUs);
168
169         lastPacketTime = millis();
170         Serial.println("Sistema listo. Esperando comandos seriales (ej: X:0.2,Y:0.0,Z
171 :0.25 o X:120,Y:0,Z:100)...");
172
173     void loop() {
174         if (isEmergency) {
175             digitalWrite(LASER_PIN, LOW);
176             delay(100);
177             return;
178         }
179
180         // Watchdog de seguridad: apaga el laser si se pierde la comunicacion serial
181         if (millis() - lastPacketTime > SERIAL_TIMEOUT_MS) {
182             digitalWrite(LASER_PIN, LOW);
183         }
184
185         if (Serial.available() > 0) {
186             String line = Serial.readStringUntil('\n');
187             line.trim();
188
189             if (line.length() > 0) {
190                 // Comando manual para parada de emergencia
191                 if (line.equalsIgnoreCase("ESTOP") || line.equalsIgnoreCase("EMERGENCY")) {
192                     triggerEmergency("Comando manual recibido");
193
194                     return;
195                 }
196
197                 int idxJ1 = line.indexOf("J1:");
198                 int idxJ2 = line.indexOf("J2:");
199                 int idxJ3 = line.indexOf("J3:");
200                 int idxL = line.indexOf("L:");
201
202                 int idxX = line.indexOf("X:");
203                 int idxY = line.indexOf("Y:");
204                 int idxZ = line.indexOf("Z:");
205
206                 if (idxJ1 != -1 && idxJ2 != -1 && idxJ3 != -1) {
207                     int endJ1 = line.indexOf(',', idxJ1);
208                     int endJ2 = line.indexOf(',', idxJ2);
209                     int endJ3 = (idxL != -1) ? idxL : line.length();
210
211                     String j1Str = line.substring(idxJ1 + 3, endJ1 != -1 ? endJ1 : line.length
212 ());
213                     String j2Str = line.substring(idxJ2 + 3, endJ2 != -1 ? endJ2 : line.length
214 ());
215                     String j3Str = line.substring(idxJ3 + 3, endJ3 != -1 ? endJ3 : line.length
216 ());
217
218                     float j1_deg = j1Str.toFloat();
219                     float j2_deg = j2Str.toFloat();
220                     float j3_deg = j3Str.toFloat();
221
222                     bool laserVal = false;
223                     if (idxL != -1) {
224                         String lStr = line.substring(idxL + 3);
225                         laserVal = (lStr.toInt() > 0);
226                     }
227
228                     float thetal = j1_deg * PI / 180.0f;
229                     float theta2 = j2_deg * PI / 180.0f;
230                     float theta3 = j3_deg * PI / 180.0f;
231
232                     int base_us = angleToMicroseconds(thetal, MIN_THETA1, MAX_THETA1);
233                     int shoulder_us = angleToMicroseconds(theta2, MIN_THETA2, MAX_THETA2);
234                     int elbow_us = angleToMicroseconds(theta3, MIN_THETA3, MAX_THETA3);
235
236                     lastPacketTime = millis();
237                     if (laserVal) {
238                         digitalWrite(LASER_PIN, HIGH);
239                     } else {
240                         digitalWrite(LASER_PIN, LOW);
241                     }
242
243                     moveToTarget(base_us, shoulder_us, elbow_us);
244
245                     Serial.print("OK | Joints set: J1=");
246                     Serial.print(j1_deg, 1);
247                     Serial.print(" J2=");
248                     Serial.print(j2_deg, 1);
249                     Serial.print(" J3=");
250                     Serial.print(j3_deg, 1);
251                     Serial.print(" | Laser=");
252                     Serial.println(laserVal ? "ON" : "OFF");
253                 } else if (idxX != -1 && idxY != -1 && idxZ != -1) {
254                     int endX = line.indexOf(',', idxX);
255                     int endY = line.indexOf(',', idxY);
256                     int endZ = line.length();
257
258                     String xStr = line.substring(idxX + 2, endX != -1 ? endX : line.length());
259                     String yStr = line.substring(idxY + 2, endY != -1 ? endY : line.length());
260                     String zStr = line.substring(idxZ + 2, endZ);
261
262                     float raw_x = xStr.toFloat();
263                     float raw_y = yStr.toFloat();
264                     float raw_z = zStr.toFloat();
265
266                     float scale = 1.0f;
267                     if (abs(raw_x) > 2.0f || abs(raw_y) > 2.0f || abs(raw_z) > 2.0f) {
268                         scale = 0.001f;
269                     }
270
271                     float x = raw_x * scale;
272                     float y = raw_y * scale;
273                     float z = raw_z * scale;
274
275                     float thetal = 0.0f, theta2 = 0.0f, theta3 = 0.0f;
276                     if (solveInverseKinematics(x, y, z, thetal, theta2, theta3)) {
277                         lastPacketTime = millis();
278                         digitalWrite(LASER_PIN, HIGH);
279
280                         int base_us = angleToMicroseconds(thetal, MIN_THETA1, MAX_THETA1);
281                         int shoulder_us = angleToMicroseconds(theta2, MIN_THETA2, MAX_THETA2);
282                         int elbow_us = angleToMicroseconds(theta3, MIN_THETA3, MAX_THETA3);
283
284                         // Movimiento de servos en pasos graduales suaves
285                         moveToTarget(base_us, shoulder_us, elbow_us);
286
287                         Serial.print("OK | Target: X=");
288                         Serial.print(x, 4);
289                         Serial.print(" Y=");
290                         Serial.print(y, 4);
291                         Serial.print(" Z=");
292                         Serial.print(z, 4);
293                         Serial.print(" | Joints: T1=");
294                         Serial.print(thetal * 180.0f / PI, 1);
295                         Serial.print(" T2=");
296                         Serial.print(theta2 * 180.0f / PI, 1);
297                         Serial.print(" T3=");
298                         Serial.println(theta3 * 180.0f / PI, 1);
299                     } else {
300                         Serial.println("ERR: Posicion cartesiana fuera del area de alcance.");
301                     }
302                 } else {
303                     Serial.println("WARN: Formato invalido. Use 'X:val,Y:val,Z:val'");
304                 }
305             }
306         }
307     }

```

```

301 }
302 }
303 }

```

Listing 10. Firmware embebido en C++ para control de servomotores y conmutación láser en el ESP32 (main.cpp).

APÉNDICE I CÓDIGO DEL ALGORITMO DE ADELGAZAMIENTO ZHANG-SUEN (WEB_BRIDGE.PY)

```

1 def zhang_suen_thinning(img):
2     import numpy as np
3     im = (img > 0).astype(np.uint8)
4     im = np.pad(im, 1, mode='constant', constant_values=0)
5
6     while True:
7         # Paso 1
8         p2 = im[:-2, 1:-1]
9         p3 = im[:-2, 2:]
10        p4 = im[1:-1, 2:]
11        p5 = im[2:, 2:]
12        p6 = im[2:, 1:-1]
13        p7 = im[2:, :-2]
14        p8 = im[1:-1, :-2]
15        p9 = im[:-2, :-2]
16
17        B = p2 + p3 + p4 + p5 + p6 + p7 + p8 + p9
18
19        A = ((p2 == 0) & (p3 == 1)).astype(np.uint8) + \
20            ((p3 == 0) & (p4 == 1)).astype(np.uint8) + \
21            ((p4 == 0) & (p5 == 1)).astype(np.uint8) + \
22            ((p5 == 0) & (p6 == 1)).astype(np.uint8) + \
23            ((p6 == 0) & (p7 == 1)).astype(np.uint8) + \
24            ((p7 == 0) & (p8 == 1)).astype(np.uint8) + \
25            ((p8 == 0) & (p9 == 1)).astype(np.uint8) + \
26            ((p9 == 0) & (p2 == 1)).astype(np.uint8)
27
28        cond1 = (B >= 2) & (B <= 6) & (A == 1) & (p2 * p4 * p6 == 0) & (p4 * p6 *
29            p8 == 0)
30        inner = im[1:-1, 1:-1]
31        to_delete_1 = (inner == 1) & cond1
32
33        im[1:-1, 1:-1] = np.where(to_delete_1, 0, inner)
34
35        # Paso 2
36        p2 = im[:-2, 1:-1]
37        p3 = im[:-2, 2:]
38        p4 = im[1:-1, 2:]
39        p5 = im[2:, 2:]
40        p6 = im[2:, 1:-1]
41        p7 = im[2:, :-2]
42        p8 = im[1:-1, :-2]
43        p9 = im[:-2, :-2]
44
45        B = p2 + p3 + p4 + p5 + p6 + p7 + p8 + p9
46        A = ((p2 == 0) & (p3 == 1)).astype(np.uint8) + \
47            ((p3 == 0) & (p4 == 1)).astype(np.uint8) + \
48            ((p4 == 0) & (p5 == 1)).astype(np.uint8) + \
49            ((p5 == 0) & (p6 == 1)).astype(np.uint8) + \
50            ((p6 == 0) & (p7 == 1)).astype(np.uint8) + \
51            ((p7 == 0) & (p8 == 1)).astype(np.uint8) + \
52            ((p8 == 0) & (p9 == 1)).astype(np.uint8) + \
53            ((p9 == 0) & (p2 == 1)).astype(np.uint8)
54
55        cond2 = (B >= 2) & (B <= 6) & (A == 1) & (p2 * p4 * p8 == 0) & (p2 * p6 *
56            p8 == 0)
57        inner = im[1:-1, 1:-1]
58        to_delete_2 = (inner == 1) & cond2
59
60        im[1:-1, 1:-1] = np.where(to_delete_2, 0, inner)
61
62        if not np.any(to_delete_1) and not np.any(to_delete_2):
63            break
64
65    return (im[1:-1, 1:-1] * 255).astype(np.uint8)

```

Listing 11. Algoritmo de esqueletización vectorizada de Zhang-Suen (web_bridge.py).

APÉNDICE J CÓDIGO DE VERIFICACIÓN CINEMÁTICA Y DH (TEST_KINEMATICS.PY)

```

1 import numpy as np
2
3 # Constantes del robot
4 d1 = 60.0
5 d2 = 36.0
6 a2 = 15.0
7 a3 = 146.0
8 a4 = 161.0
9 d4 = -7.0
10
11 phi2 = 1.030041
12 phi3 = -0.191392
13
14 def dh_matrix(theta, d, a, alpha):
15     alpha_rad = np.radians(alpha) if isinstance(alpha, (int, float)) else alpha
16     return np.array([
17         [np.cos(theta), -np.sin(theta)*np.cos(alpha_rad), np.sin(theta)*np.sin(
18             alpha_rad), a*np.cos(theta)],

```

```

18         [np.sin(theta), np.cos(theta)*np.cos(alpha_rad), -np.cos(theta)*np.sin(
19             alpha_rad), a*np.sin(theta)],
20         [0, np.sin(alpha_rad), np.cos(alpha_rad),
21             d],
22         [0, 0, 0, 1]
23     ])
24
25 def fk_dh(theta1, theta2, theta3):
26     theta2_star = theta2 + phi2
27     theta3_star = theta3 + phi3 - phi2
28
29     T1 = dh_matrix(theta1, d1 + d2, a2, 90)
30     T2 = dh_matrix(theta2_star, 0, a3, 0)
31     T3 = dh_matrix(theta3_star, d4, a4, 0)
32
33     T = T1 @ T2 @ T3
34     return T
35
36 def fk_analytical(theta1, theta2, theta3):
37     theta2_star = theta2 + phi2
38     theta3_star = theta3 + phi3 - phi2
39
40     x_plane = a2 + a3 * np.cos(theta2_star) + a4 * np.cos(theta2_star + theta3_star)
41     z_plane = d1 + d2 + a3 * np.sin(theta2_star) + a4 * np.sin(theta2_star +
42         theta3_star)
43
44     x = x_plane * np.cos(theta1) + d4 * np.sin(theta1)
45     y = x_plane * np.sin(theta1) - d4 * np.cos(theta1)
46     z = z_plane
47     return np.array([x, y, z])
48
49 # Probar con angulos cero
50 t1, t2, t3 = 0.0, 0.0, 0.0
51 T_dh = fk_dh(t1, t2, t3)
52 pos_dh = T_dh[:3, 3]
53 pos_analyt = fk_analytical(t1, t2, t3)
54
55 print("D-H End Effector Position (3 matrices):")
56 print(pos_dh)
57 print("\nAnalytical End Effector Position:")
58 print(pos_analyt)

```

Listing 12. Comparación de modelos de cinemática directa y Denavit-Hartenberg (test_kinematics.py).

APÉNDICE K CÓDIGO DE VALIDACIÓN DE ALINEACIÓN DE MARCOS TF (TEST_TF_ALIGNMENT.PY)

```

1 #!/usr/bin/env python3
2 import rclpy
3 from rclpy.node import Node
4 import tf2_ros
5 import math
6 import numpy as np
7
8 class TFAlignmentValidator(Node):
9     def __init__(self):
10         super().__init__('tf_alignment_validator')
11         self.tf_buffer = tf2_ros.Buffer()
12         self.tf_listener = tf2_ros.TransformListener(self.tf_buffer, self)
13
14         # Esperar un poco a que se llene el buffer de TFs
15         self.timer = self.create_timer(1.0, self.validate_alignment)
16         self.get_logger().info("Validator node started. Waiting for TF data...")
17
18     def validate_alignment(self):
19         self.timer.cancel()
20
21         frames = ['system1', 'system2', 'system3', 'flange']
22         base_frame = 'link1'
23
24         print("\n" + "="*70)
25         print(" VALIDACION DE ALINEACION DE EJES X (VISTA LATERAL / PLANO X-Z de link1)")
26         print("="*70)
27
28         all_parallel = True
29
30         for frame in frames:
31             try:
32                 # Obtener la transformacion de link1 al sistema
33                 now = rclpy.time.Time()
34                 trans = self.tf_buffer.lookup_transform(base_frame, frame, now,
35                     timeout=rclpy.duration.Duration(seconds=2.0))
36
37                 # Obtener cuaternión
38                 q = trans.transform.rotation
39
40                 # Convertir cuaternión a matriz de rotación
41                 R = self.quaternion_to_matrix(q.x, q.y, q.z, q.w)
42
43                 # El eje X del frame en coordenadas de link1 es la primera columna
44                 de R
45                 x_axis = R[:, 0]
46
47                 # Calcular angulo en el plano vertical X-Z de link1
48                 angle_rad = math.atan2(x_axis[2], x_axis[0])
49                 angle_deg = math.degrees(angle_rad)
50
51                 print(f"\nFrame: {frame}")
52                 print(f" Vector eje X (en frame link1): [{x_axis[0]:.6f}, {x_axis
53                     [1]:.6f}, {x_axis[2]:.6f}])")

```

```

51         print(f" Desviacion lateral (Y): {x_axis[1]:.6f} (debe ser aprox
52         0)")
53         print(f" Angulo en plano X-Z (Pitch): {angle_deg:.4f} deg")
54         if abs(x_axis[1]) > 0.005:
55             print(" [WARN] ADVERTENCIA: El eje X no esta en el mismo plano
56             de movimiento (Y != 0)")
57             all_parallel = False
58             elif abs(angle_deg) > 0.1:
59                 print(f" [WARN] ADVERTENCIA: El eje X no es paralelo al eje X
60                 horizontal (Angulo = {angle_deg:.4f} deg)")
61                 all_parallel = False
62             else:
63                 print(" [OK] ALINEACION PERFECTA (eje X paralelo y horizontal)
64                 ")
65             except Exception as e:
66                 self.get_logger().error(f"No se pudo obtener la transformacion para
67                 {frame}: {str(e)}")
68                 all_parallel = False
69             print("\n" + "="*70)
70             if all_parallel:
71                 print(" RESULTADO FINAL: [OK] TODOS LOS EJES X SON PARALELOS Y
72                 HORIZONTALES!")
73                 print(" El brazo esta perfectamente recto y alineado en la pose cero de
74                 la simulacion.")
75             else:
76                 print(" RESULTADO FINAL: [ERR] EXISTEN DESALINEACIONES O DESVIACIONES
77                 FUERA DE LIMITES.")
78                 print("="*70 + "\n")
79             rclpy.shutdown()
80
81     def quaternion_to_matrix(self, x, y, z, w):
82         return np.array([
83             [1 - 2*(y**2 + z**2), 2*(x*y - w*z), 2*(x*z + w*y)],
84             [2*(x*y + w*z), 1 - 2*(x**2 + z**2), 2*(y*z - w*x)],
85             [2*(x*z - w*y), 2*(y*z + w*x), 1 - 2*(x**2 + y**2)]
86         ])
87
88 def main(args=None):
89     rclpy.init(args=args)
90     node = TFAlignmentValidator()
91     try:
92         rclpy.spin(node)
93     except SystemExit:
94         pass
95
96 if __name__ == '__main__':
97     main()

```

Listing 13. Validación de paralelismo de ejes longitudinales en base al árbol de transformadas TF (test_tf_alignment.py).

APÉNDICE L

CÓDIGO DE VERIFICACIÓN ANALÍTICA DE URDF (TEST_URDF_TF_ALIGNMENT.PY)

```

1  #!/usr/bin/env python3
2  import math
3  import numpy as np
4
5  def rpy_to_matrix(r, p, y):
6      cr, sr = math.cos(r), math.sin(r)
7      cp, sp = math.cos(p), math.sin(p)
8      cy, sy = math.cos(y), math.sin(y)
9
10     Rx = np.array([[1, 0, 0], [0, cr, -sr], [0, sr, cr]])
11     Ry = np.array([[cp, 0, sp], [0, 1, 0], [-sp, 0, cp]])
12     Rz = np.array([[cy, -sy, 0], [sy, cy, 0], [0, 0, 1]])
13
14     return Rz @ Ry @ Rx
15
16 # Datos de origen de las articulaciones de la simulacion (URDF actual)
17 R_w_base = rpy_to_matrix(1.57079632679, 0, 0)
18 R_base_l1 = rpy_to_matrix(-1.57079632679, 0, 0)
19 R_l1_l2 = rpy_to_matrix(1.57079632679, 0, 0)
20 R_l2_l3 = rpy_to_matrix(0, 0, 0)
21 R_l3_flange = rpy_to_matrix(0, 0, 0)
22
23 # Marcos de referencia visuales (visualize_frame) en el URDF actual
24 R_base_sys0 = rpy_to_matrix(-1.57079632679, 0, 0)
25 R_l2_sys1 = rpy_to_matrix(0, 0, 0)
26 R_l3_sys2 = rpy_to_matrix(0, 0, 0)
27 R_flange_sys3 = rpy_to_matrix(0, 0, 0)
28
29 transforms = {
30     "system1": R_l1_l2 @ R_l2_sys1,
31     "system2": R_l1_l2 @ R_l2_l3 @ R_l3_sys2,
32     "system3": R_l1_l2 @ R_l2_l3 @ R_l3_flange @ R_flange_sys3,
33     "flange": R_l1_l2 @ R_l2_l3 @ R_l3_flange
34 }
35
36 print("\n" + "="*75)
37 print(" CALCULO ANALITICO DE ALINEACION DE EJES X (DESDE LA SIMULACION URDF)")
38 print(" Plano de referencia: vista lateral (plano X-Z de link1)")
39 print("="*75)
40
41 all_parallel = True
42
43 for name, R_l1_S in transforms.items():
44     x_axis_in_l1 = R_l1_S[:, 0]
45     y_deviation = x_axis_in_l1[1]
46     angle_rad = math.atan2(x_axis_in_l1[2], x_axis_in_l1[0])
47     angle_deg = math.degrees(angle_rad)

```

```

48     print(f"\nSistema: {name}")
49     print(f" Direccion del eje X en link1: [{x_axis_in_l1[0]:.6f}, {x_axis_in_l1
50     [1]:.6f}, {x_axis_in_l1[2]:.6f}])")
51     print(f" Desviacion lateral (Y): {y_deviation:.6f} (debe ser 0 para estar
52     coplanar)")
53     print(f" Angulo lateral (Pitch): {angle_deg:.4f} deg (debe ser 0 deg para
54     estar horizontal)")
55     if abs(y_deviation) > 1e-5:
56         print(" [ERR] ERROR: El eje X se sale del plano vertical (desviacion Y no
57         es cero).")
58         all_parallel = False
59     elif abs(angle_deg) > 1e-5:
60         print(f" [ERR] ERROR: El eje X no es horizontal en la vista lateral (
61         Angulo = {angle_deg:.4f} deg).")
62         all_parallel = False
63     else:
64         print(" [OK] PERFECTO: El eje X es coplanar, paralelo y horizontal.")
65
66 print("\n" + "="*75)
67 if all_parallel:
68     print(" VERIFICACION EXITOSA: [OK] TODOS LOS EJES X SON PARALELOS Y
69     PERFECTAMENTE ALINEADOS!")
70 else:
71     print(" VERIFICACION FALLIDA: [ERR] Existen desalineaciones entre los marcos.")
72 print("="*75 + "\n")

```

Listing 14. Verificación analítica de la rotación y coplanaridad de las articulaciones en el modelo URDF (test_urdf_tf_alignment.py).

APÉNDICE M

CÓDIGO DE LECTURA DE DISTANCIAS DE MARCOS TF EN GAZEBO (GET_TF_DISTANCES.PY)

```

1  #!/usr/bin/env python3
2  import rclpy
3  from rclpy.node import Node
4  import tf2_ros
5  import sys
6
7  class TFEchoAll(Node):
8      def __init__(self):
9          super().__init__('tf_echo_all')
10         self.tf_buffer = tf2_ros.Buffer()
11         self.tf_listener = tf2_ros.TransformListener(self.tf_buffer, self)
12         self.timer = self.create_timer(1.0, self.query_all_tfs)
13         self.run_once = False
14
15     def query_all_tfs(self):
16         if self.run_once:
17             return
18
19         frames = ['link1', 'link2', 'link3', 'flange']
20         base = 'base_link'
21
22         print("\n== COORDENADAS DE LAS ARTICULACIONES DESDE TF EN GAZEBO ==")
23         print(f"Sistema de referencia base: {base}\n")
24
25         success = True
26         for frame in frames:
27             try:
28                 trans = self.tf_buffer.lookup_transform(base, frame, rclpy.time.
29                 Time())
30                 translation = trans.transform.translation
31                 rotation = trans.transform.rotation
32                 x, y, z, w = rotation.x, rotation.y, rotation.z, rotation.w
33
34                 print(f"-> {frame} respecto a {base}:")
35                 print(f" Traslacion: [{translation.x:.6f}, {translation.y:.6f}, {
36                 translation.z:.6f}] m")
37                 print(f" [{translation.x*1000:.3f}, {translation.y
38                 *1000:.3f}, {translation.z*1000:.3f}] mm")
39                 print(f" Rotacion (Quaternion): xyzw = [{x:.6f}, {y:.6f}, {z:.6f
40                 }, {w:.6f}]\n")
41
42             except Exception as e:
43                 print(f"Error buscando transformada de {base} a {frame}: {str(e)}")
44                 success = False
45
46         if success:
47             self.run_once = True
48             sys.exit(0)
49
50 def main(args=None):
51     rclpy.init(args=args)
52     node = TFEchoAll()
53     try:
54         rclpy.spin(node)
55     except SystemExit:
56         pass
57     except KeyboardInterrupt:
58         pass
59     finally:
60         node.destroy_node()
61         rclpy.shutdown()
62
63 if __name__ == '__main__':
64     main()

```

Listing 15. Script de lectura y visualización de transformadas de articulaciones desde TF en Gazebo (get_tf_distances.py).

APÉNDICE N

CÓDIGO DEL PARSER Y ANALIZADOR DE PARÁMETROS XACRO (GET_GAZEBO_DISTANCES.PY)

```
1 import xml.etree.ElementTree as ET
2 import math
3 import os
4
5 def parse_urdf():
6     urdf_path = "/home/reve/Documents/UC/robotica/finalProject/ros2_ws/src/
7         arm_simulation/urdf/arm.urdf.xacro"
8     if not os.path.exists(urdf_path):
9         print(f"Error: No se encontro el archivo URDF en {urdf_path}")
10        return
11
12    tree = ET.parse(urdf_path)
13    root = tree.getroot()
14
15    print("=== DISTANCIAS Y LONGITUDES EXTRAIDAS DEL MODELO URDF EN GAZEBO ===\n")
16
17    joint_world = root.find(".//joint[@name='world_to_base']")
18    if joint_world is not None:
19        origin = joint_world.find("origin")
20        print(f"world_to_base:")
21        print(f"    xyz: {origin.attrib.get('xyz')}")
22        print(f"    rpy: {origin.attrib.get('rpy')}\n")
23
24    joint1 = root.find(".//joint[@name='joint1']")
25    if joint1 is not None:
26        origin = joint1.find("origin")
27        print(f"joint1 (Base a Link1):")
28        print(f"    xyz: {origin.attrib.get('xyz')} (Desfase vertical base height d1
29            = 60 mm)")
30        print(f"    rpy: {origin.attrib.get('rpy')}\n")
31
32    joint2 = root.find(".//joint[@name='joint2']")
33    if joint2 is not None:
34        origin = joint2.find("origin")
35        xyz = [float(x) for x in origin.attrib.get('xyz').split()]
36        print(f"joint2 (Link1 a Hombro):")
37        print(f"    xyz: {origin.attrib.get('xyz')} (Desfase radial a2 = {xyz
38            [0]*1000:.3f} mm, Desfase vertical d2 = {xyz[2]*1000:.3f} mm)")
39        print(f"    rpy: {origin.attrib.get('rpy')}\n")
40
41    joint3 = root.find(".//joint[@name='joint3']")
42    if joint3 is not None:
43        origin = joint3.find("origin")
44        xyz = [float(x) for x in origin.attrib.get('xyz').split()]
45        length = math.sqrt(xyz[0]**2 + xyz[1]**2) * 1000
46        angle = math.atan2(xyz[1], xyz[0]) * 180 / math.pi
47        print(f"joint3 (Hombro a Codo):")
48        print(f"    xyz: {origin.attrib.get('xyz')}")
49        print(f"    longitud del brazo (Link2) a3: {length:.3f} mm")
50        print(f"    Angulo de inclinacion phi2: {angle:.6f} deg ((math.atan2(xyz[1],
51            xyz[0]):.6f) rad)\n")
52
53    joint_flange = root.find(".//joint[@name='joint_3-flange']")
54    if joint_flange is not None:
55        origin = joint_flange.find("origin")
56        xyz = [float(x) for x in origin.attrib.get('xyz').split()]
57        length = math.sqrt(xyz[0]**2 + xyz[1]**2) * 1000
58        angle = math.atan2(xyz[1], xyz[0]) * 180 / math.pi
59        print(f"joint_3-flange (Codo a Breda):")
60        print(f"    xyz: {origin.attrib.get('xyz')}")
61        print(f"    Longitud del antebrazo (Link3) a4: {length:.3f} mm")
62        print(f"    Angulo de inclinacion phi3: {angle:.6f} deg ((math.atan2(xyz[1],
63            xyz[0]):.6f) rad)")
64        print(f"    Desfase transversal d4: {xyz[2]*1000:.3f} mm\n")
65
66    if __name__ == "__main__":
67        parse_urdf()
```

Listing 16. Script parser XML para extraer dimensiones y ángulos del archivo URDF/Xacro (get_gazebo_distances.py).

REFERENCIAS

- [1] J. Craig, *Introduction to Robotics: Mechanics and Control*, 4th ed. Pearson, 2017.
- [2] I. Hinojosa, “Repositorio de Código del Proyecto FP_Robotics”, GitHub, 2026. Disponible en: https://github.com/Reve7339/FP_Robotics.